

Contents

1	Activity Diagram - Admin Model Moderation	5
1.1	Overview	5
1.2	Swim Lanes	5
1.3	Process Flow	6
1.4	Decision Points Summary	6
1.5	Notes	6
2	Activity Diagram - Admin User Moderation	6
2.1	Overview	7
2.2	Swim Lanes	7
2.3	Process Flow	7
2.4	Decision Points Summary	8
2.5	Notes	8
3	Activity Diagram - Analytics	8
3.1	Overview	8
3.2	Swim Lanes	8
3.3	Process Flow	8
3.4	Decision Points Summary	10
3.5	Notes	10
4	Activity Diagram - Cart Add	10
4.1	Overview	10
4.2	Swim Lanes	10
4.3	Process Flow	11
4.4	Decision Points Summary	11
4.5	Key Implementation Details	11
5	Activity Diagram - Catalog Filter	11
5.1	Overview	11
5.2	Swim Lanes	14
5.3	Process Flow	14
5.4	Decision Points Summary	14
5.5	Notes	14
6	Activity Diagram - Checkout	14
6.1	Overview	16
6.2	Swim Lanes	16
6.3	Process Flow	16
6.4	Decision Points Summary	16
6.5	Key Implementation Details	16
7	Activity Diagram - Community Feed	17
7.1	Overview	17
7.2	Swim Lanes	18
7.3	Process Flow	18
7.4	Decision Points Summary	18
7.5	Key Implementation Details	18
8	Activity Diagram - Complete Profile	19
8.1	Overview	19
8.2	Swim Lanes	19
8.3	Process Flow	19

8.4	Decision Points Summary	21
8.5	Notes	21
9	Activity Diagram - Dexie Picks	21
9.1	Overview	21
9.2	Swim Lanes	21
9.3	Process Flow	21
9.4	Decision Points Summary	23
9.5	Key Implementation Details	23
10	Activity Diagram - Dexie Tagline	23
10.1	Overview	23
10.2	Swim Lanes	25
10.3	Process Flow	25
10.4	Decision Points Summary	25
10.5	Key Implementation Details	26
11	Activity Diagram - Follow Toggle	26
11.1	Overview	27
11.2	Swim Lanes	27
11.3	Process Flow	27
11.4	Decision Points Summary	27
11.5	Key Implementation Details	28
12	Activity Diagram - Google OAuth	28
12.1	Overview	29
12.2	Swim Lanes	29
12.3	Process Flow	29
12.4	Decision Points Summary	29
12.5	Notes	30
13	Activity Diagram - Local Login	30
13.1	Overview	30
13.2	Swim Lanes	30
13.3	Process Flow	30
13.4	Decision Points Summary	32
13.5	Relationship to Sequence Diagram	32
13.6	Notes	32
14	Activity Diagram - Model Delete	32
14.1	Overview	32
14.2	Swim Lanes	32
14.3	Process Flow	33
14.4	Decision Points Summary	34
14.5	Key Implementation Details	34
15	Activity Diagram - Model Download	34
15.1	Overview	35
15.2	Swim Lanes	35
15.3	Process Flow	35
15.4	Decision Points Summary	35
15.5	Key Implementation Details	36
16	Activity Diagram - Model Review	36
16.1	Overview	36

16.2 Swim Lanes	36
16.3 Process Flow	36
16.4 Decision Points Summary	38
16.5 Key Implementation Details	38
17 Activity Diagram - Model Upload	38
17.1 Overview	38
17.2 Swim Lanes	39
17.3 Process Flow	39
17.4 Decision Points Summary	39
17.5 Notes	39
18 Activity Diagram - Notifications	40
18.1 Overview	40
18.2 Swim Lanes	40
18.3 Process Flow	40
18.4 Decision Points Summary	41
18.5 Key Implementation Details	41
19 Activity Diagram - Order Cancel	41
19.1 Overview	41
19.2 Swim Lanes	41
19.3 Process Flow	43
19.4 Decision Points Summary	43
19.5 Key Implementation Details	43
20 Activity Diagram - Payment Webhook	43
20.1 Overview	44
20.2 Swim Lanes	44
20.3 Process Flow	44
20.4 Decision Points Summary	45
20.5 Key Implementation Details	45
21 Activity Diagram - Post Create	45
21.1 Overview	47
21.2 Swim Lanes	47
21.3 Process Flow	47
21.4 Decision Points Summary	47
21.5 Key Implementation Details	48
22 Activity Diagram - Post Delete	48
22.1 Overview	48
22.2 Swim Lanes	49
22.3 Process Flow	49
22.4 Decision Points Summary	49
22.5 Key Implementation Details	49
23 Activity Diagram - Post Like	50
23.1 Overview	50
23.2 Swim Lanes	50
23.3 Process Flow	50
23.4 Decision Points Summary	51
23.5 Key Implementation Details	51
24 Activity Diagram - Print Job Lifecycle	51

24.1	Overview	51
24.2	Swim Lanes	51
24.3	Process Flow	52
24.3.1	Phase 1: Accept Job	52
24.3.2	Phase 2: Ship Job	53
24.3.3	Phase 3: Deliver Job	53
24.4	Decision Points Summary	53
24.5	Key Implementation Details	53
25	Activity Diagram - Print Order	53
25.1	Overview	55
25.2	Swim Lanes	55
25.3	Process Flow	55
25.4	Decision Points Summary	55
25.5	Key Implementation Details	56
26	Activity Diagram - Profile Update	56
26.1	Overview	57
26.2	Swim Lanes	57
26.3	Process Flow	57
26.4	Decision Points Summary	57
26.5	Key Implementation Details	58
27	Activity Diagram - User Registration	58
27.1	Overview	58
27.2	Swim Lanes	58
27.3	Process Flow	59
27.4	Decision Points Summary	59
27.5	Notes	59
28	Activity Diagram - Report	59
28.1	Overview	60
28.2	Swim Lanes	60
28.3	Process Flow	60
28.4	Decision Points Summary	61
28.5	Notes	61
29	Activity Diagram - Session Hydration	61
29.1	Overview	61
29.2	Swim Lanes	61
29.3	Process Flow	63
29.4	Decision Points Summary	63
29.5	Notes	63
30	Activity Diagram - Stats Aggregation	63
30.1	Overview	63
30.2	Swim Lanes	64
30.3	Process Flow	64
30.4	Decision Points Summary	65
30.5	Notes	65
31	Activity Diagram - User Review	65
31.1	Overview	65
31.2	Swim Lanes	65
31.3	Process Flow	65

31.3.1 Phase 1: Eligibility Check	65
31.3.2 Phase 2: Review Submission	67
31.4 Decision Points Summary	67
31.5 Key Implementation Details	67
32 Activity Diagram - Wishlist Toggle	67
32.1 Overview	68
32.2 Swim Lanes	68
32.3 Process Flow	68
32.4 Decision Points Summary	69
32.5 Key Implementation Details	69

1 Activity Diagram - Admin Model Moderation

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_admin_model_moderation

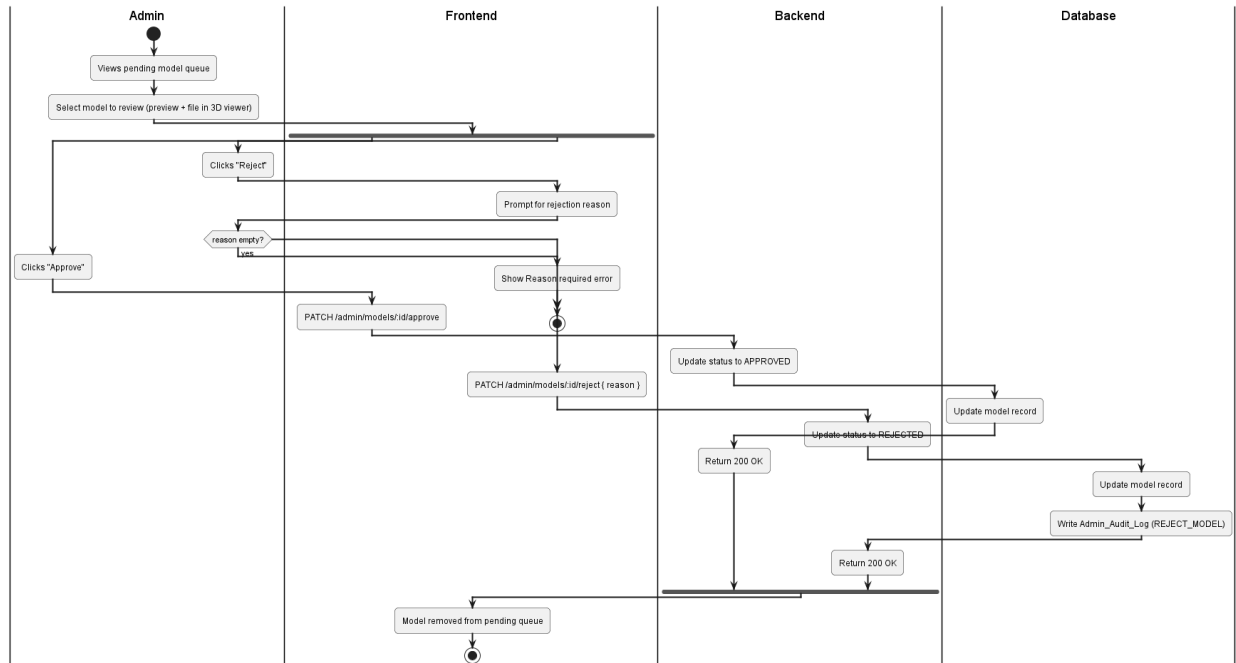


Figure 1: Activity Diagram - Admin Model Moderation

1.1 Overview

This activity diagram shows the model moderation process using a parallel fork to represent the two mutually exclusive admin decisions: approve or reject.

1.2 Swim Lanes

Lane	Responsibility
Admin	Navigates the review queue and makes decisions

Lane	Responsibility
Frontend	Renders the queue, prompts for reasons, sends requests
Backend	Updates model status and writes audit logs
Database	Persists changes

1.3 Process Flow

1. **[Admin]** Views the pending model queue on the admin dashboard.
2. **[Admin]** Selects a model to review (previews the 3D file in the viewer).
3. **[Frontend]** Displays action buttons: “Approve” and “Reject”.

Fork - Admin chooses one of:

Approve path:

4A. **[Admin]** Clicks “Approve”. 5A. **[Frontend]** Sends PATCH /admin/models/:id/approve. 6A. **[Backend]** Updates the model: `status = APPROVED`. 7A. **[Database]** Updates model record. 8A. **[Backend]** Returns 200 OK.

Reject path:

4B. **[Admin]** Clicks “Reject”. 5B. **[Frontend]** Prompts the Admin to enter a rejection reason. 6B. **[Admin]** Validates: if reason empty, shows error and stops. 7B. **[Frontend]** Sends PATCH /admin/models/:id/reject { reason }. 8B. **[Backend]** Updates the model: `status = REJECTED`. 9B. **[Database]** Updates model record AND writes Admin_Audit_Log (REJECT_MODEL). 10B. **[Backend]** Returns 200 OK.

Rejoin:

9. **[Frontend]** Model is removed from the pending queue.
-

1.4 Decision Points Summary

Decision	Yes Path	No Path
Admin chooses Approve?	Approve path	-
Admin chooses Reject?	Reject path	-
Rejection reason empty?	Show error, stop	Submit

1.5 Notes

- Approval does not write an audit log. Rejection does. This asymmetry is intentional.
- The “reason required” validation is enforced at the Frontend level (prompt) and should also be validated on the Backend before the database write.

2 Activity Diagram - Admin User Moderation

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_admin_user

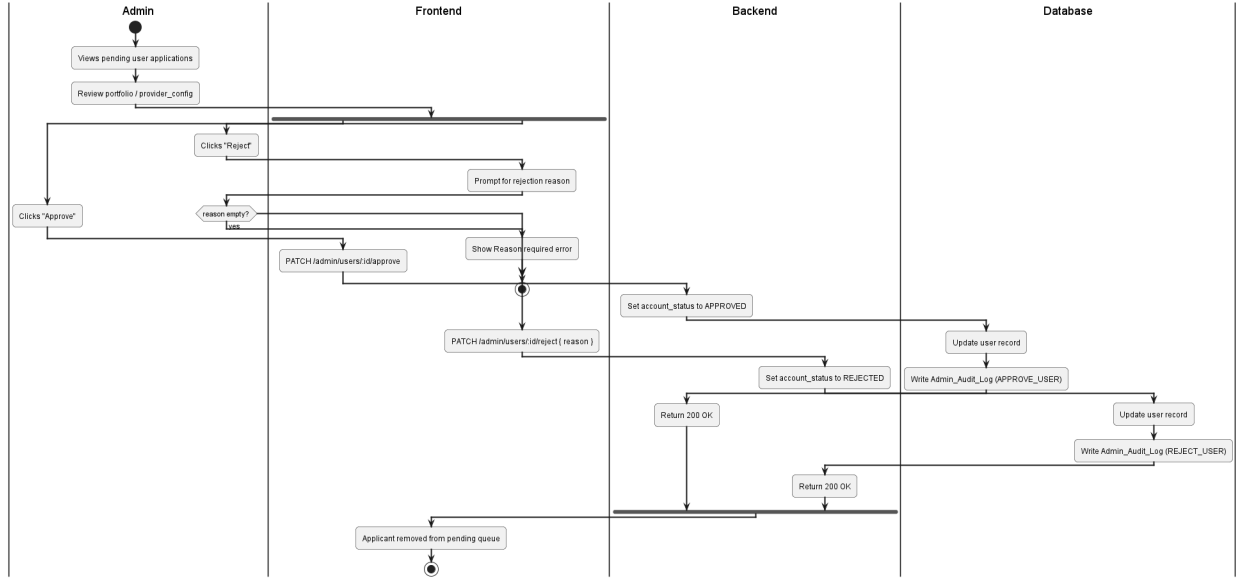


Figure 2: Activity Diagram - Admin User Moderation

2.1 Overview

This activity diagram shows the same approve/reject pattern applied to Artist and Provider account applications, with parallel fork notation for the two mutually exclusive admin decisions.

2.2 Swim Lanes

Lane	Responsibility
Admin	Reviews applications and makes decisions
Frontend	Renders the application queue and sends requests
Backend	Updates user status
Database	Persists changes and writes audit logs

2.3 Process Flow

1. [Admin] Views the pending user applications list.
2. [Admin] Reviews the applicant's portfolio or `provider_config`.
3. [Frontend] Displays "Approve" and "Reject" action buttons.

Fork:

Approve path:

4A. [Admin] Clicks "Approve". 5A. [Frontend] Sends `PATCH /admin/users/:id/approve`. 6A. [Backend] Sets `account_status = APPROVED`. 7A. [Database] Updates user record AND writes `Admin_Audit_Log (APPROVE_USER)`. 8A. [Backend] Returns 200 OK.

Reject path:

4B. [Admin] Clicks "Reject". 5B. [Frontend] Prompts for a rejection reason. 6B. [Admin] If reason is empty: shows error and stops. 7B. [Frontend] Sends `PATCH /admin/users/:id/reject { reason`

}. 8B. **[Backend]** Sets `account_status = REJECTED`. 9B. **[Database]** Updates user record AND writes `Admin_Audit_Log (REJECT_USER)`. 10B. **[Backend]** Returns 200 OK.

Rejoin:

9. **[Frontend]** Applicant is removed from the pending queue.

2.4 Decision Points Summary

Decision	Yes Path	No Path
Admin chooses Approve?	Approve path	-
Admin chooses Reject?	Reject path	-
Rejection reason empty?	Show error, stop	Submit

2.5 Notes

- Unlike model moderation, both approval AND rejection write audit log entries for user moderation. This reflects the higher accountability required for decisions affecting user roles.
- The `status_history` JSON array on the `User` record is also updated at each approval/rejection, providing a full history of status changes.

3 Activity Diagram - Analytics

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_analytics.

3.1 Overview

This activity diagram shows how both Artists and Admins access their respective analytics dashboards. The key difference is the role-based access gate and the scope of data returned.

3.2 Swim Lanes

Lane	Responsibility
Admin or Artist	The actor requesting analytics
Frontend	Navigates to the analytics page and renders charts
Backend	Validates role and aggregates data
Database	Executes parallel aggregation queries

3.3 Process Flow

1. **[Admin or Artist]** Opens the analytics page.
2. **[Frontend]** Sends `GET /analytics/artist-stats` (for Artist) or `GET /admin/analytics` (for Admin).
3. **[Backend]** Validates the user's role matches the endpoint.

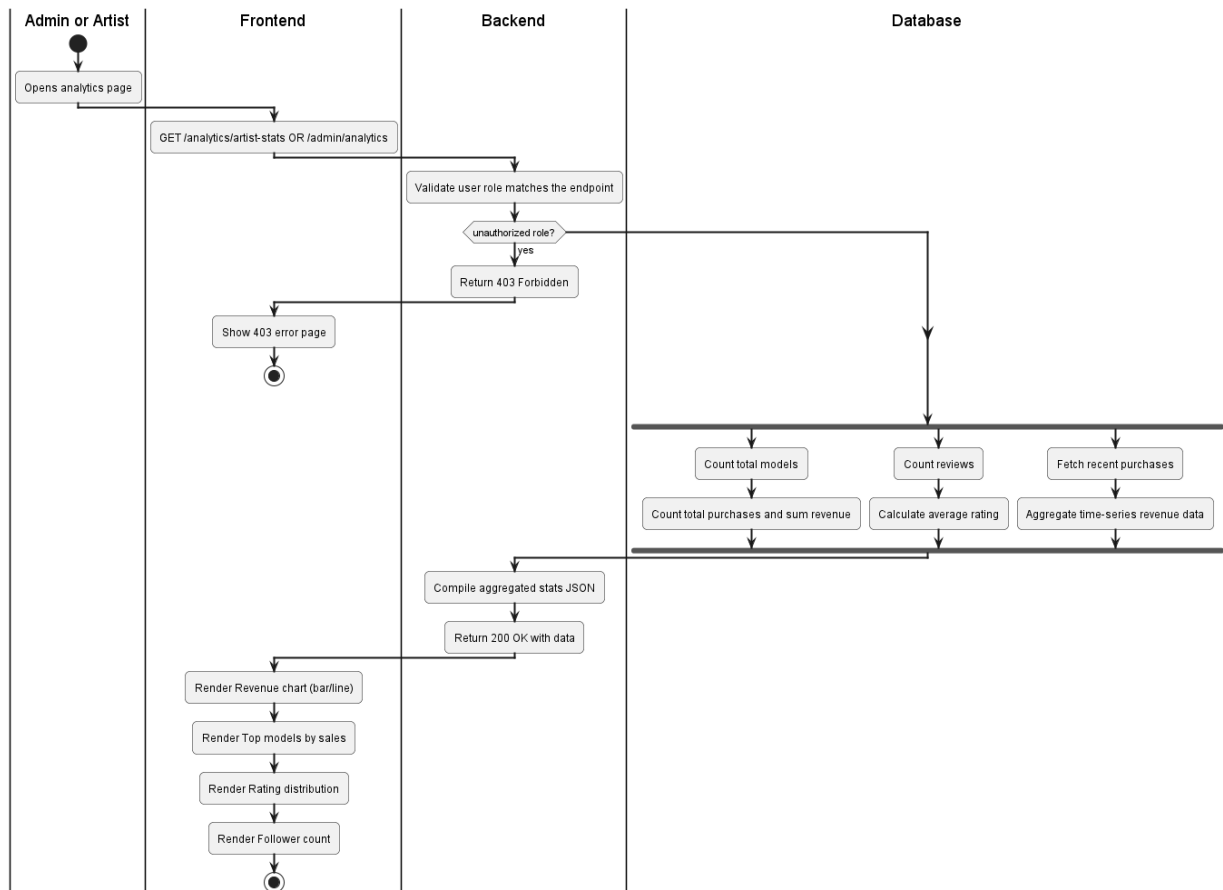


Figure 3: Activity Diagram - Analytics

- If unauthorized role: returns 403 Forbidden. Frontend shows error page. Flow ends.
4. **[Database]** Executes parallel aggregation queries:
 - Count total models (filtered by artist for Artist, platform-wide for Admin)
 - Count total purchases and sum revenue
 - Count reviews and calculate average rating
 - Fetch recent purchases and aggregate time-series revenue data
 - Find top-selling models
 - Find top-earning artists (Admin only)
 5. **[Backend]** Compiles all results into a single aggregated stats JSON.
 6. **[Backend]** Returns 200 with the data.
 7. **[Frontend]** Renders:
 - Revenue bar/line chart
 - Top models by sales
 - Rating distribution
 - Follower count (Artist view)

3.4 Decision Points Summary

Decision	Yes Path	No Path
Role matches endpoint?	Continue	Return 403

3.5 Notes

- Artist analytics are scoped to that artist's own models and sales. Admin analytics are platform-wide.
- The diagram shows the Database lane with parallel fork notation, indicating that the aggregation queries should ideally be executed concurrently. In the current Prisma implementation, these are likely sequential unless wrapped in `Promise.all`.
- The Frontend renders different chart types depending on the data returned. The backend response shape is the same for both roles; the difference is in the data scope.

4 Activity Diagram - Cart Add

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_cart

4.1 Overview

This activity diagram describes the process of adding an item to the shopping cart, detailing the validations for item status, price constraints, and deduplication of existing digital items.

4.2 Swim Lanes

Lane	Responsibility
Customer	Requests addition to cart
Frontend	Issues add-to-cart API call
Backend	Runs business rules and model validations

Lane	Responsibility
Database	Performs lookups and upsert operations

4.3 Process Flow

1. [Customer] Clicks “Add to Cart” on a model details page.
2. [Frontend] Dispatches POST /cart with { model_id } and authentication headers.
3. [Database] Looks up the model by its unique ID.
4. [Backend] Checks if the model exists and is approved (`status == APPROVED`).
 - If not: returns 404/400. Flow ends.
5. [Backend] Checks model price. If `price == 0`:
 - Returns 400 Bad Request (“Free models cannot be added to cart; download directly”). Flow ends.
6. [Database] Checks if a `Cart_Item` already exists for this (`user_id`, `model_id`) combination.
7. [Database] Executes database upsert:
 - **If exists:** Leaves quantity at 1 (digital products do not allow multiple quantities).
 - **If not exists:** Creates a new `Cart_Item` record with quantity 1.
8. [Backend] Returns 201 Created or 200 OK with the cart item.
9. [Frontend] Increments the cart item counter in the global navigation bar.

4.4 Decision Points Summary

Decision	Path/Condition	Outcome
Model exists & APPROVED?	No	Return 404 or 400, stop
Model price > 0?	No	Return 400 Bad Request, stop
Already in cart?	Yes	Perform idempotent upsert (quantity remains 1)

4.5 Key Implementation Details

- **Price Guard:** Free assets (`price == 0`) bypass the cart mechanism. They are meant to be downloaded directly. The cart system enforces this on the server side.
- **Deduplication:** Unlike physical goods, digital assets cannot be bought multiple times in the same order. Thus, `quantity` is constrained strictly to 1.
- **Database Unique Constraint:** An upsert or unique constraint on (`user_id`, `model_id`) inside the `Cart_Item` table ensures structural integrity even if concurrent clicks dispatch multiple requests.

5 Activity Diagram - Catalog Filter

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_catalog

5.1 Overview

This activity diagram details the WHERE clause construction logic for the catalog listing endpoint, showing exactly how each filter parameter affects the database query.

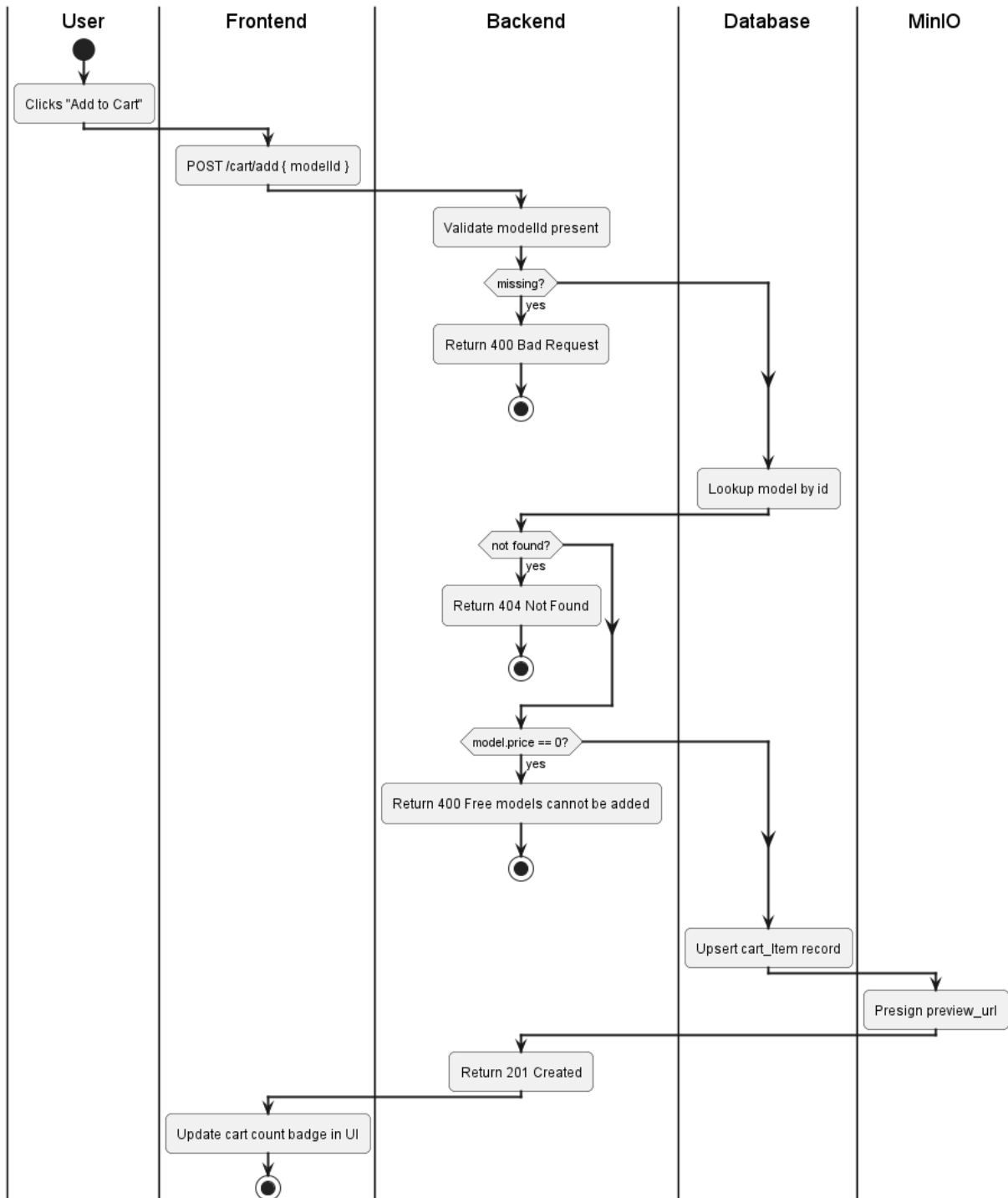


Figure 4: Activity Diagram - Cart Add

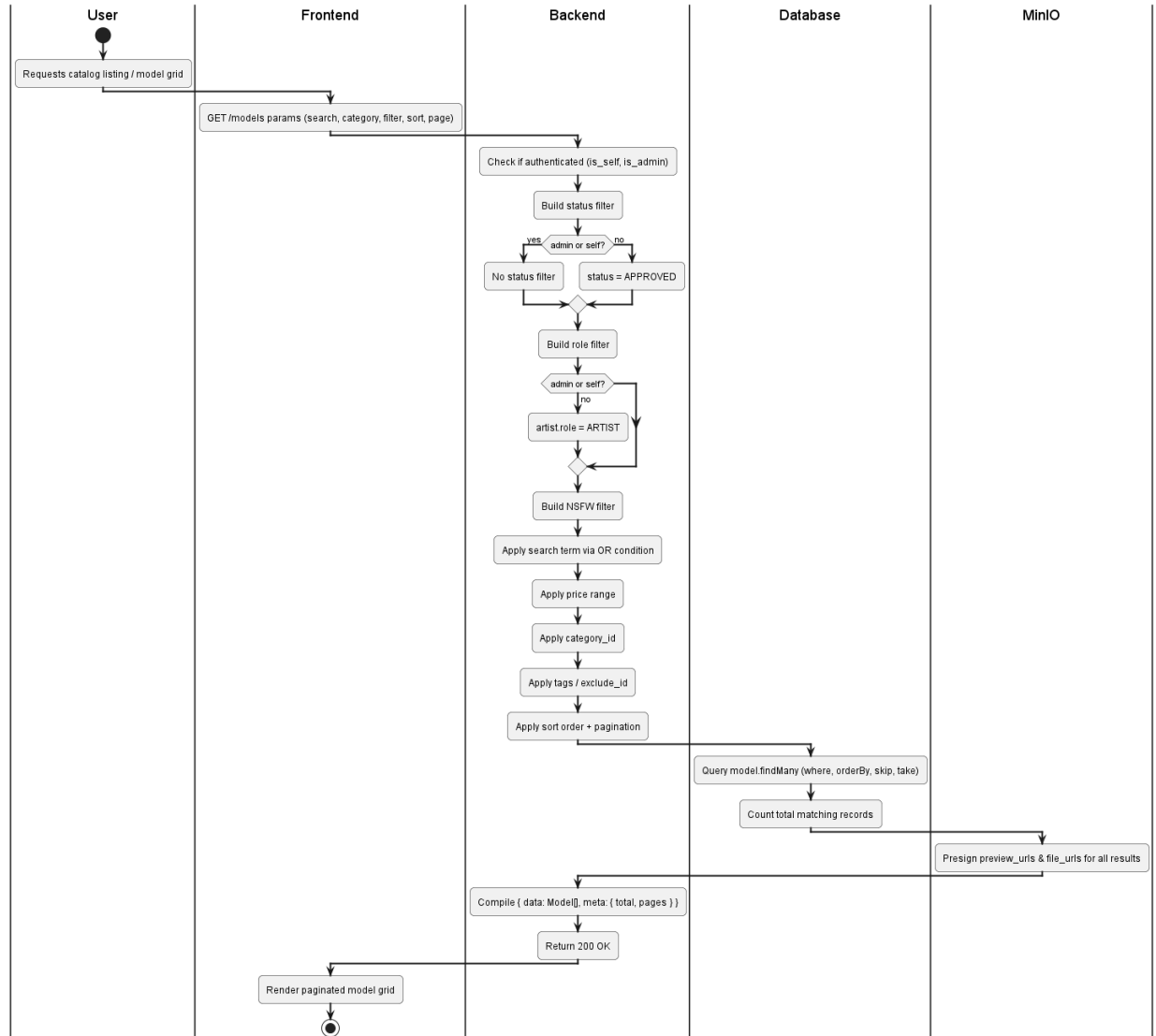


Figure 5: Activity Diagram - Catalog Filter

5.2 Swim Lanes

Lane	Responsibility
User	Initiates a catalog request
Frontend	Sends the parameterized request
Backend	Constructs the query dynamically
Database	Executes the query
MinIO	Signs preview URLs

5.3 Process Flow

1. **[User]** Requests the catalog or model grid.
2. **[Frontend]** Sends GET /models with params: `search`, `category`, `filter`, `sort`, `page`.
3. **[Backend]** Checks caller identity (`is_admin`, `is_self`).

Status filter construction: 4. **[Backend]** If admin or self-view: no status filter (all statuses visible). Otherwise: `status = APPROVED`.

Role filter construction: 5. **[Backend]** If not admin and not self: adds `artist.role = ARTIST` to ensure only artist-uploaded models are shown.

Other filters applied in sequence: 6. **[Backend]** Build NSFW filter based on `show_nsfw` parameter. 7. **[Backend]** Apply search term via OR across title, description, tags, category. 8. **[Backend]** Apply price range (`min_price` / `max_price`). 9. **[Backend]** Apply `category_id`. 10. **[Backend]** Apply tags inclusion / `exclude_id`. 11. **[Backend]** Apply sort order and pagination (`skip`, `take`).

Execution: 12. **[Database]** Executes `model.findMany(where, orderBy, skip, take)`. 13. **[Database]** Counts total matching records for pagination metadata. 14. **[MinIO]** Presigns `preview_url` and `file_url` for each result. 15. **[Backend]** Compiles `{ data: Model[], meta: { total, pages } }`. 16. **[Backend]** Returns 200 OK. 17. **[Frontend]** Renders paginated model grid.

5.4 Decision Points Summary

Decision	Yes Path	No Path
Caller is admin or self?	No status filter	<code>status = APPROVED</code>
Not admin and not self?	Add artist role filter	No role filter

5.5 Notes

- The filter chain is additive. Each parameter appends to the WHERE clause.
- The NSFW filter respects both the user's personal preference (`show_nsfw`) and the model's `is_nsfw` flag.
- The `exclude_id` parameter allows the frontend to exclude the currently viewed model from “similar models” sections on detail pages.

6 Activity Diagram - Checkout

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_checkout

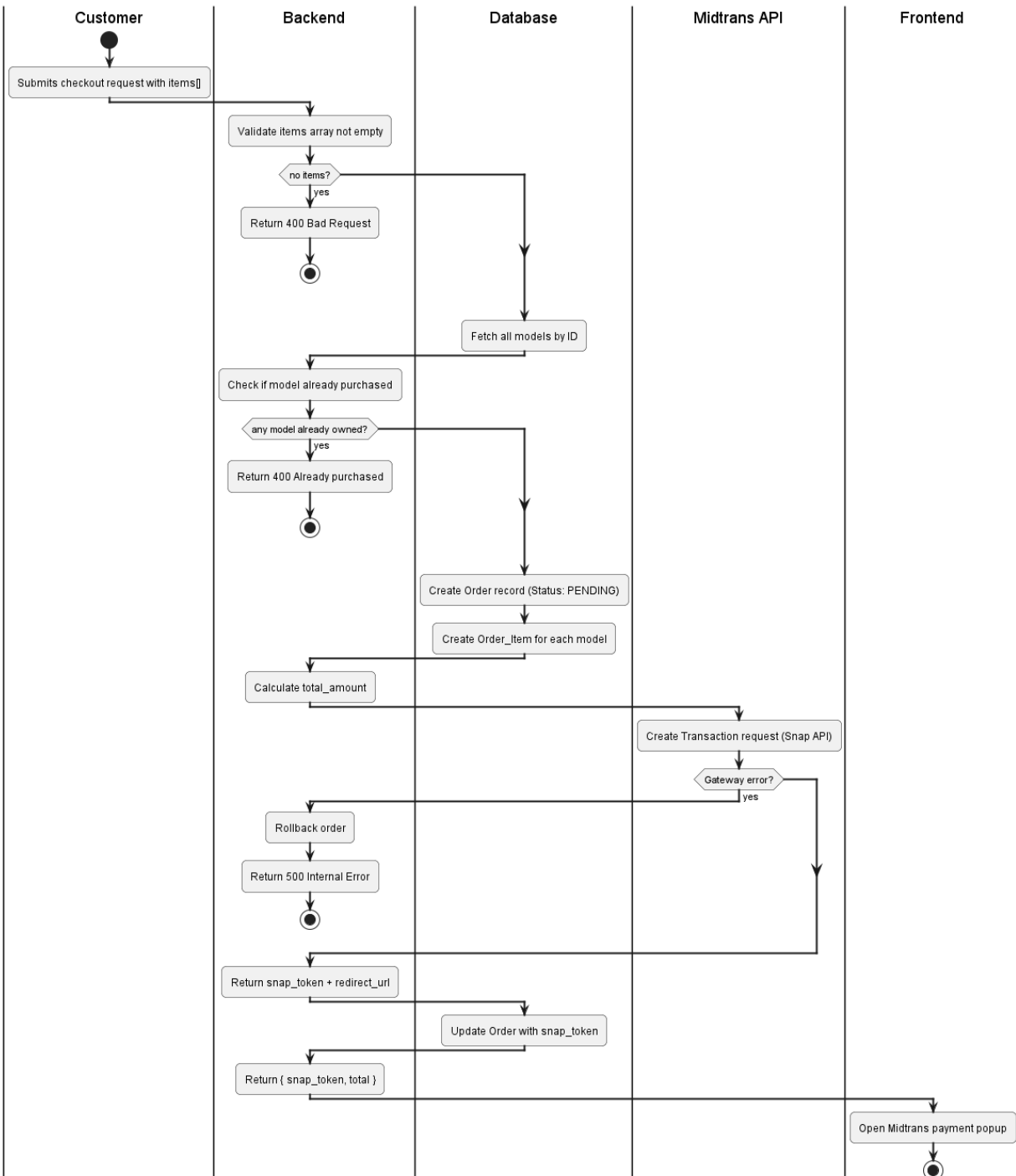


Figure 6: Activity Diagram - Checkout

6.1 Overview

This activity diagram covers the checkout flow from initiating order validation to generating a payment gateway token and displaying the Midtrans payment popup.

6.2 Swim Lanes

Lane	Responsibility
Customer	Initiates checkout and pays
Frontend	Issues API request and renders Midtrans Snap iframe
Backend	Runs safety checks, aggregates prices, and creates orders
Database	Performs purchase validation and creates Order record
Midtrans API	Registers transaction and issues Snap tokens

6.3 Process Flow

1. **[Customer]** Clicks “Checkout” inside the Cart page.
 2. **[Frontend]** Dispatches POST `/orders/checkout` with `{ items: [{ model_id, quantity }] }`.
 3. **[Database]** Looks up the selected models.
 4. **[Backend]** Validates all items are available and **APPROVED**.
 5. **[Database]** Queries **Purchase** history to ensure none of these items are already owned by the user.
 - If a duplicate purchase is detected: returns 400 Bad Request. Flow ends.
 6. **[Database]** Computes the sum of all model prices and inserts a new **Order** record:
 - `status = PENDING`
 - `type = ASSET`
 - `total_amount = sum(prices)`
 7. **[Backend]** Initiates request to **[Midtrans API]** Snap endpoint with order metadata and gross amount.
 8. **[Midtrans API]** Registers the transaction and issues a unique `{ snap_token, redirect_url }`.
 9. **[Database]** Updates the **Order** record, writing the retrieved `snap_token` and `snap_redirect_url`.
 10. **[Backend]** Returns 200 OK along with the token and order summaries.
 11. **[Frontend]** Triggers the global window script `snap.pay(token)`.
 12. **[Customer]** Sees the interactive payment modal overlay (Midtrans Snap widget).
-

6.4 Decision Points Summary

Decision	Path/Condition	Outcome
Items exist & APPROVED?	No	Return 404/400, stop
Already purchased?	Yes	Return 400 Bad Request, stop

6.5 Key Implementation Details

- **Duplicate Check:** To prevent users from buying items they already own, the checkout endpoint performs a strict verification against the **Purchase** table. If any item in the cart is already purchased, checkout is blocked.

- **Midtrans Snap Integration:** The transaction is registered with Midtrans *before* the customer pays. The resulting token is saved in the database, allowing the user to resume payment if the popup is closed.
- **Transactional Consistency:** If the Midtrans API call fails, the local **Order** record remains in PENDING state or can be rolled back.

7 Activity Diagram - Community Feed

UML Type: Activity Diagram Category: Social Source: docs/diagrams/activity/social/activity_community_

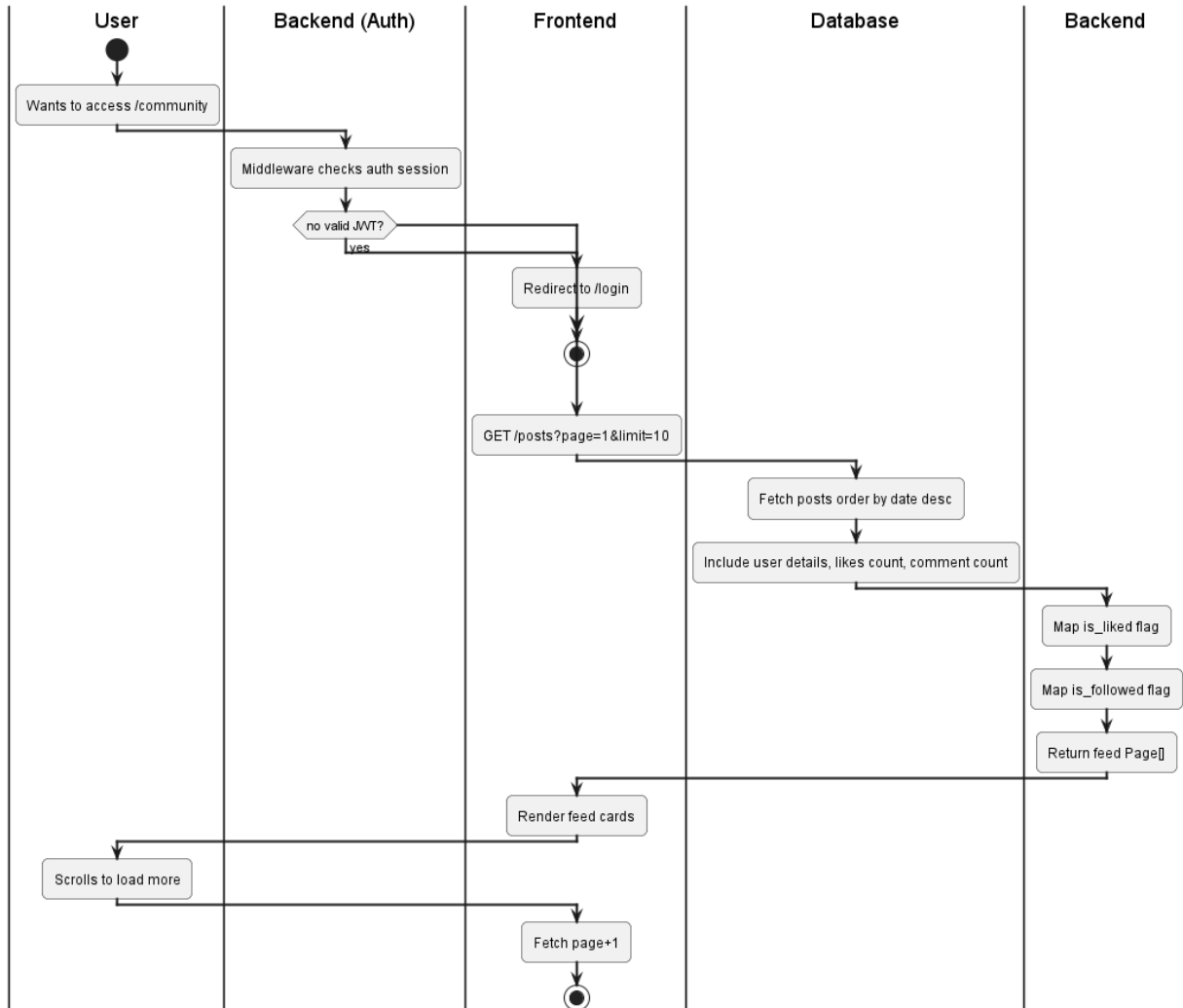


Figure 7: Activity Diagram - Community Feed

7.1 Overview

This activity diagram describes the process of loading and rendering the community feed. It focuses on the server-side operations that assemble social posts and dynamically inject user-specific social states (such as whether the current user has liked a post or followed its author).

7.2 Swim Lanes

Lane	Responsibility
User	Requests the community page
Frontend	Issues API calls and renders post widgets
Backend	Assembles search queries and maps custom flags
Database	Queries posts, comments, likes, and follows

7.3 Process Flow

1. **[User]** Navigates to the Community Feed page.
2. **[Frontend]** Dispatches `GET /posts` with the user's session token.
3. **[Database]** Queries `Post` records, ordering by `created_at` descending.
4. **[Database]** Fetches related entity counts in the query:
 - Count of `Post_Like` records per post.
 - Count of `Post_Comment` records per post.
5. **[Backend]** Verifies the requesting user session status (authenticated vs. guest).

Conditional Logic Branching:

- **If User is Authenticated:**
 1. **[Database]** Queries `Post_Like` for records matching (`user_id`, `post_id`) to determine like status.
 2. **[Database]** Queries `Follow` for records matching (`follower_id`, `post.user_id`) to determine follow status.
 3. **[Backend]** Injects dynamic flags per post: `is_liked = true/false`, `is_followed = true/false`.
 - **If User is a Guest (Anonymous):**
 1. **[Backend]** Sets flags to defaults: `is_liked = false`, `is_followed = false`.
6. **[Backend]** Returns 200 OK along with the compiled posts payload.
 7. **[Frontend]** Renders the post cards, using the dynamic flags to style the like (heart) and follow buttons.

7.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	Yes	Query DB for likes and follows, inject dynamic boolean flags
Authenticated?	No	Set like/follow flags to false

7.5 Key Implementation Details

- **Server-Side Mapping:** Mapping flags like `is_liked` and `is_followed` directly on the server reduces frontend logic and prevents a flurry of additional API calls from the client to determine interaction states.

- **Denormalized Optimization:** Although comment and like counts can be calculated dynamically, the backend utilizes denormalized counters (`like_count` and `comment_count`) on the `Post` table to keep list fetches highly performant.

8 Activity Diagram - Complete Profile

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_complete_profile

8.1 Overview

This activity diagram describes the profile completion step that new Google OAuth users must complete before using the platform. After registering via Google, users are redirected here to choose a permanent username.

8.2 Swim Lanes

Lane	Responsibility
User	The new Google OAuth user
Frontend	Enforces the redirect and renders the form
Backend	Validates and saves the chosen username
Database	Checks uniqueness and persists the update

8.3 Process Flow

1. **[Frontend]** After Google OAuth login with `needs_username = true`, redirects to `/complete-profile`.
 2. **[User]** Sees the “Choose a username” form.
 3. **[Frontend]** Checks auth session. If not authenticated or `needs_username = false`: redirects away. Flow ends.
 4. **[User]** Enters desired username.
 5. **[Frontend]** Client validates:
 - Username is 3-30 characters, alphanumeric, no spaces.
 - If invalid: shows inline error. User corrects.
 6. **[Frontend]** Sends `PATCH /users/complete-profile { username }`.
 7. **[Backend]** Validates:
 - Username is present.
 - Matches allowed pattern (3-30 chars, `^[a-zA-Z0-9_]+$`).
 - If invalid: returns 400.
 8. **[Database]** Checks username uniqueness.
 - If taken (P2002): Backend returns 409 “Username already taken”.
 9. **[Database]** Updates user record with the chosen username.
 10. **[Backend]** Signs a fresh JWT and sets a new `3dex_session` cookie.
 11. **[Backend]** Returns 200 with updated user.
 12. **[Frontend]** Hydrates `AuthContext`, redirects to home. User can now use the platform.
-

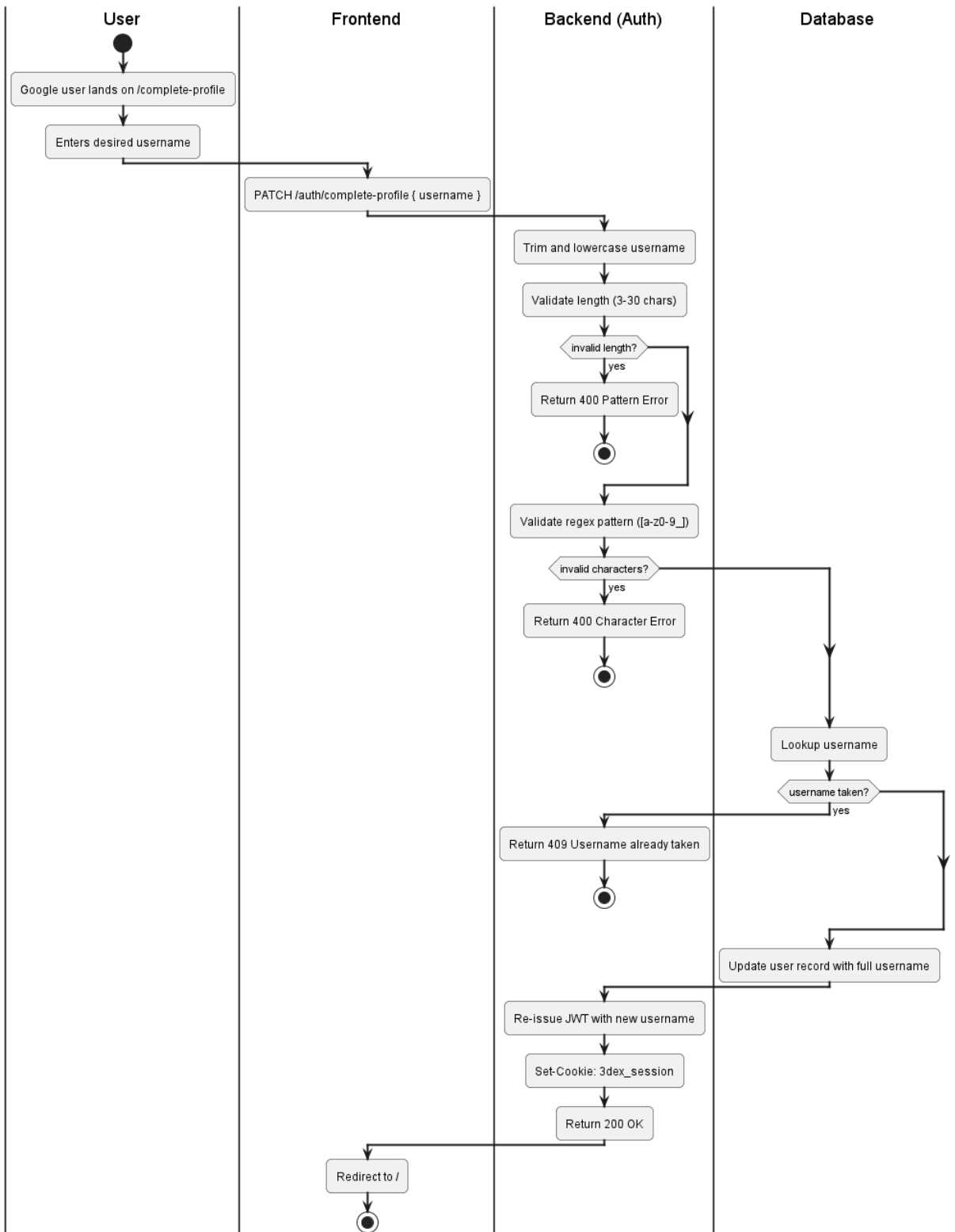


Figure 8: Activity Diagram - Complete Profile

8.4 Decision Points Summary

Decision	Yes Path	No Path
Session valid and needs_username?	Show form	Redirect away
Client validation passes?	Submit	Show error, retry
Server validation passes?	Continue	Return 400
Username unique?	Save	Return 409, retry

8.5 Notes

- This is a mandatory step for Google OAuth new users only. Local registration users choose their username during registration.
- After this step, `needs_username` becomes irrelevant since the user now has a permanent username.
- The fresh JWT issued at the end updates the cookie so subsequent `GET /auth/me` calls return the correct username.
- The page redirects away if the user already has a username, preventing repeated visits to the complete-profile form.

9 Activity Diagram - Dexie Picks

UML Type: Activity Diagram **Category:** Dexie **Source:** docs/diagrams/activity/dexie/activity_dexie_picks

9.1 Overview

This activity diagram describes the personalized product recommendation flow powered by Dexie AI. It details the process of collecting user interactions (likes, purchases, wishlist items), generating semantic search query embeddings, and querying PostgreSQL with cosine similarity to retrieve personalized recommendations.

9.2 Swim Lanes

Lane	Responsibility
Customer	Navigates to the home page or catalog feed
Frontend	Issues recommendations retrieval API call
Backend	Retrieves taste vectors and constructs queries
Embedding Service	Converts interaction keywords into a continuous space vector
Database	Executes high-performance cosine similarity queries

9.3 Process Flow

1. **[Customer]** Accesses the home screen or personalized dashboard.
2. **[Frontend]** Dispatches `GET /dexie/picks?limit=8` with user headers.
3. **[Backend]** Verifies the requesting user session.

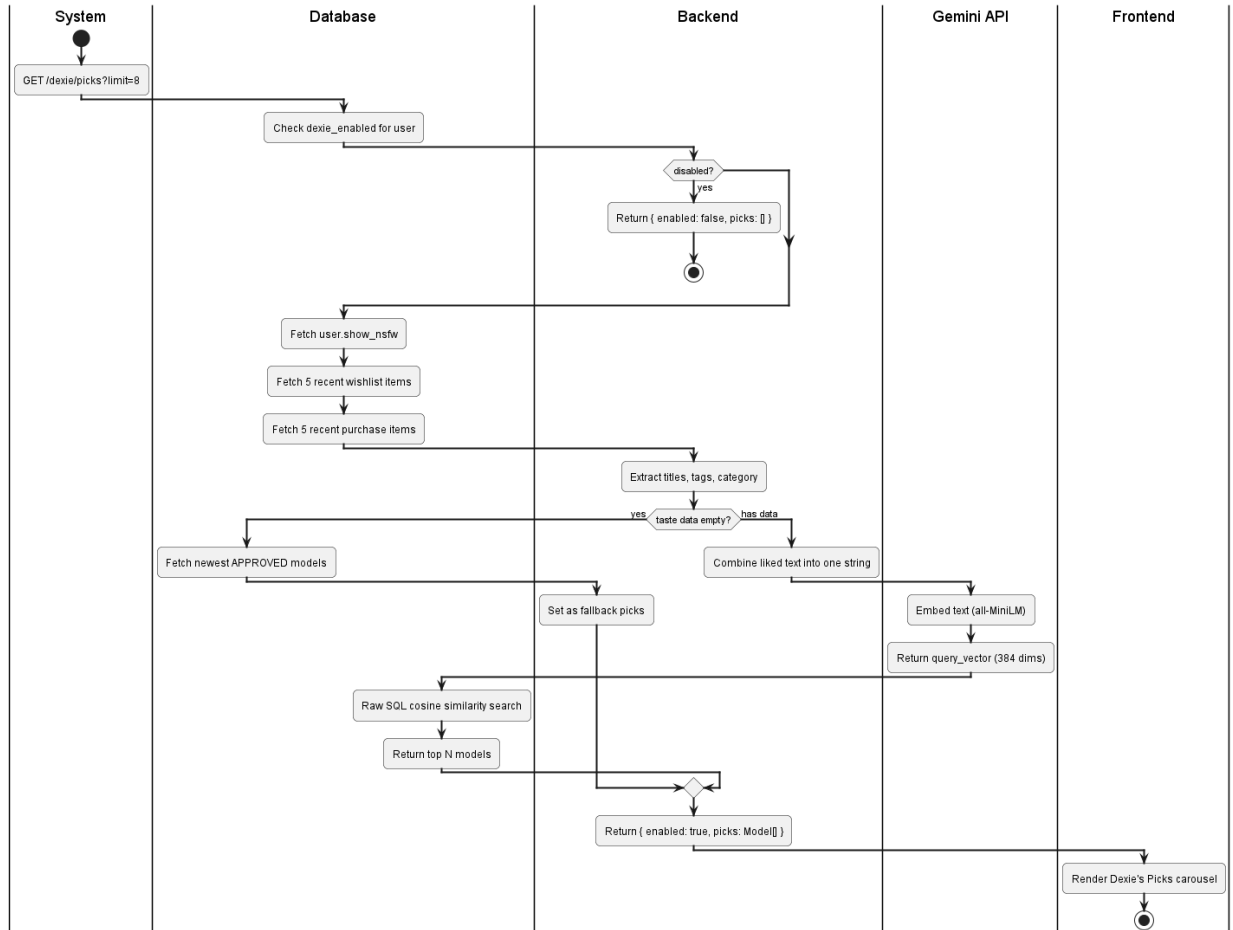


Figure 9: Activity Diagram - Dexie Picks

- If user is anonymous (guest): Backend queries for the newest 8 approved models as general fallbacks and skips personal embedding paths. Flow rejoins at step 8.
4. **[Database]** Queries for the user’s recent interactions:
 - Fetches the last 5 wishlisted items (including tags and categories).
 - Fetches the last 5 purchased items (including tags and categories).
 5. **[Backend]** Checks if any interaction history exists.
 - **No history found:** Queries for the newest 8 approved models as fallbacks. Flow rejoins at step 8.
 - **History exists:** Combines all tags and categories from the historical items into a single, clean text string.
 6. **[Embedding Service]** Converts the compiled interaction keywords into a 384-dimensional dense vector using the `all-MiniLM-L6-v2` transformer model.
 7. **[Database]** Executes a raw SQL parameter-safe query comparing the query vector against the pre-computed `model.embedding` column using PostgreSQL’s cosine distance operator (`<=>`).
 8. **[Database]** Retrieves the top 8 models with the smallest cosine distance (highest semantic similarity) that have `status == APPROVED` and do not belong to the user themselves.
 9. **[Backend]** Returns 200 OK along with the recommendations payload.
 10. **[Frontend]** Renders the curated products in the “Dexie’s Picks” carousel.

9.4 Decision Points Summary

Decision	Path/Condition	Outcome
Logged in?	No	Load newest 8 models as general fallbacks
Taste data exists?	No	Load newest 8 models as fallbacks
Taste data exists?	Yes	Generate embedding, execute raw pgvector cosine similarity

9.5 Key Implementation Details

- **Raw SQL Cosine Similarity:** Because standard Prisma queries do not support PostgreSQL `<=>` vector distance operators, the database interaction uses the `prisma.$queryRaw` helper to execute the comparison securely.
- **Cold Start Recovery:** By checking for empty interaction arrays and automatically reverting to a chronological fallback queue, the recommendation system is safe against cold-start bottlenecks.

10 Activity Diagram - Dexie Tagline

UML Type: Activity Diagram **Category:** Dexie **Source:** docs/diagrams/activity/dexie/activity_dexie_tagli

10.1 Overview

This activity diagram describes the process of generating contextual AI taglines (assistant speech bubbles) when a user navigates the platform. It maps the integration of user settings checks, multi-tier caching (Redis/Memory), Gemini Flash API fallback flows, and frontend display deduplication.

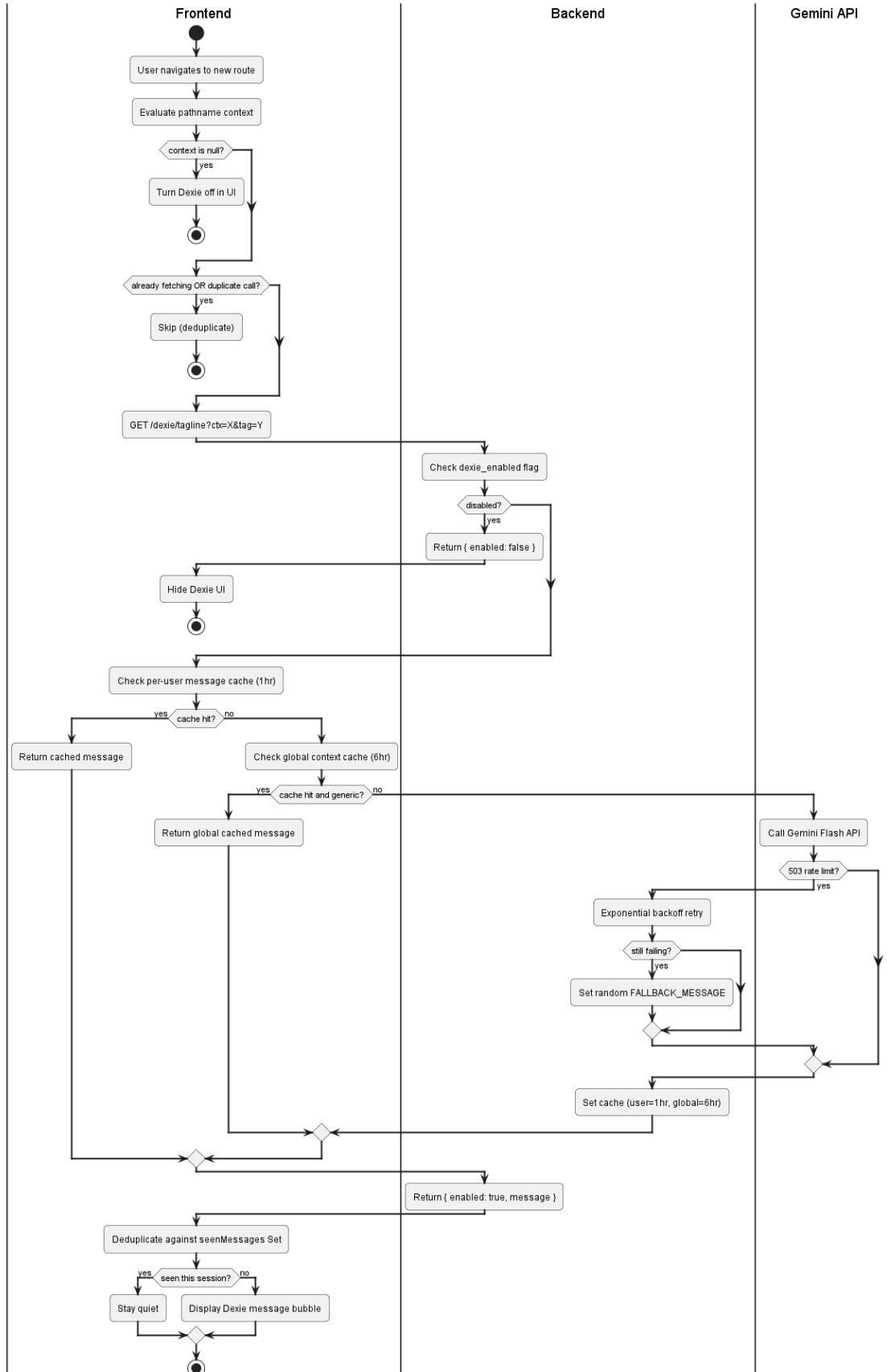


Figure 10: Activity Diagram - Dexie Tagline

10.2 Swim Lanes

Lane	Responsibility
User	Navigates the application
Frontend	Tracks path changes, sends requests, and filters repetitive dialogue
Backend	Verifies feature flags, routes cache lookups, and manages fallback states
Cache Service	Resolves per-user and global AI tagline keys
Gemini API	Generates personalized text dynamically

10.3 Process Flow

1. [User] Navigates to a platform path (e.g., /catalog/:id).
2. [Frontend] Identifies path change and constructs a context coordinate: { ctx: "catalog", tag: model_id }.
3. [Frontend] Dispatches request: GET /dexie/tagline?ctx=catalog&tag=model_id.
4. [Backend] Queries user.dexie_enabled.
 - If disabled: returns { enabled: false, message: null }. Flow ends.
5. [Cache Service] Looks up the per-user contextual tagline cache key (1-hour TTL).

Cache Evaluation:

- **Path A: Cache Hit**
 1. Cache Service returns the saved tagline directly. Rejoins at step 9.
- **Path B: Cache Miss**
 1. Backend checks the global contextual cache key (6-hour TTL).
 2. If global cache hits: Returns global tagline. Rejoins at step 9.
 3. If global cache misses: Backend initiates request to [Gemini API] with custom context prompts.

Gemini API Call and Fallback Evaluation:

- **If Gemini returns 200 OK:**
 - Saves the resulting string to the per-user cache (1-hour TTL) and global cache (6-hour TTL). Rejoins at step 9.
- **If Gemini returns 503 (Rate Limit or Error):**
 - Backend runs exponential backoff retries (up to 2 times).
 - If retries fail: Resolves a random, statically stored fallback tagline from the local library. Rejoins at step 9.

Rejoin and Frontend Handshake:

6. [Backend] Returns 200 OK with the finalized tagline.
7. [Frontend] Receives the tagline and compares it against seenMessages in local state.
 - If already seen in the current session: Hides bubble. Flow ends.
 - If completely new: Adds message to seenMessages, sets local state, and plays the tagline animation overlay.

10.4 Decision Points Summary

Decision	Path/Condition	Outcome
Dexie enabled?	No	Return enabled: false, stop
User Cache Hit?	Yes	Return cached tagline, stop
Global Cache Hit?	Yes	Return cached tagline, stop
Gemini API status?	Fail after retries	Return random fallback tagline
Message already seen?	Yes	Hide bubble, stop

10.5 Key Implementation Details

- **Two-Tier Cache Strategy:** A per-user cache protects the system from repeated calls when the same user navigates back and forth. A global cache protects the system by sharing generic page tagline prompts (like home page welcoming lines) across different users.
- **Session Deduplication:** The frontend keeps a tracking state (`seenMessages`) which is initialized as an empty Set when the user mounts the app. This prevents the AI assistant from showing the same pop-up repeatedly, which would lead to poor user experience.

11 Activity Diagram - Follow Toggle

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_follow_toggle

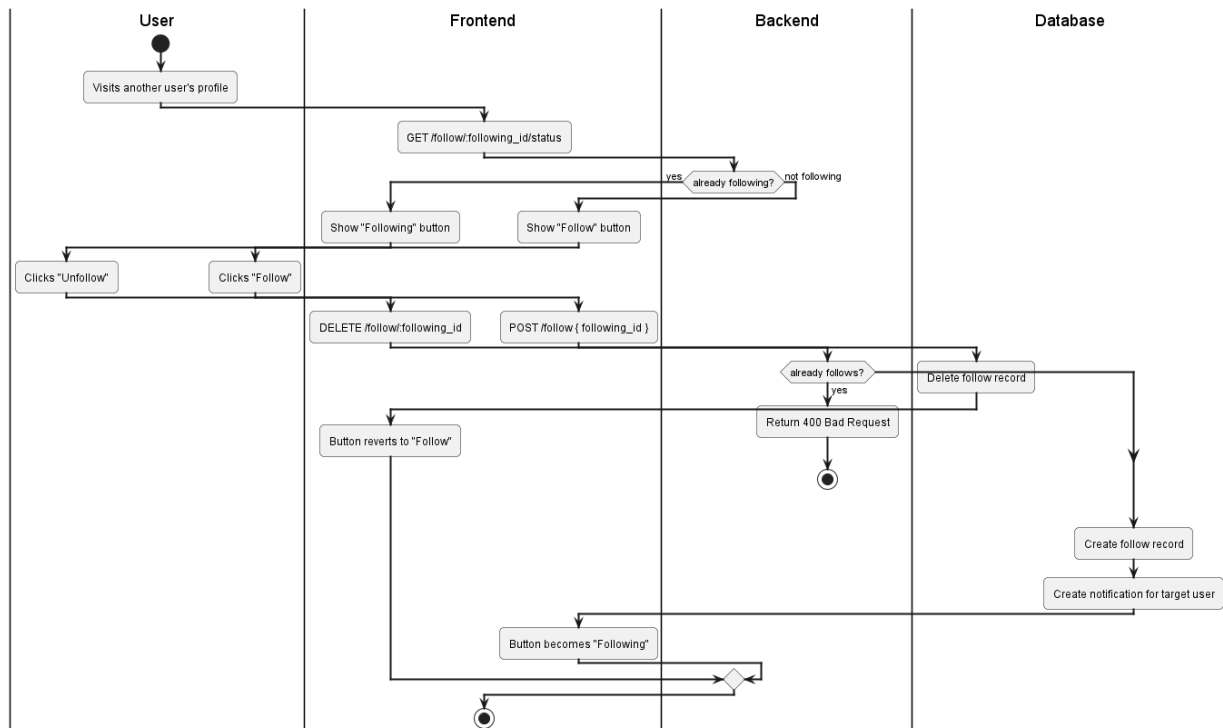


Figure 11: Activity Diagram - Follow Toggle

11.1 Overview

This activity diagram describes the dual-path follow/unfollow toggle flow. It details how the backend handles session validation, checks follow states in the database, and dispatches in-app notifications only on fresh follow events.

11.2 Swim Lanes

Lane	Responsibility
User	Clicks the follow/unfollow toggle button
Frontend	Sends the toggle request and handles UI states
Backend	Runs credentials checks and triggers notifications
Database	Performs lookups, inserts, and deletions
Notification Service	Sends alerts to the target user

11.3 Process Flow

1. [User] Clicks the “Follow” or “Unfollow” button on a profile page.
2. [Frontend] Dispatches `POST /social/follow/toggle` with `{ target_user_id }`.
3. [Backend] Verifies the requesting user is authenticated.
 - If not authenticated: returns 401 Unauthorized. Flow ends.
4. [Backend] Verifies `user_id != target_user_id` (users cannot follow themselves).
 - If attempting self-follow: returns 400 Bad Request. Flow ends.
5. [Database] Queries `Follow` table for a record matching the unique pair (`follower_id`, `following_id`).
6. [Backend] Evaluates the check:

Decision Branch: Follow Record Exists?

- **Path A: Record Exists (Unfollow)**
 1. [Database] Deletes the `Follow` record.
 2. [Backend] Returns 200 OK with `{ following: false }`.
 3. [Frontend] Reverts button text to “Follow”.
 - **Path B: Record Does Not Exist (Follow)**
 1. [Database] Inserts a new `Follow` record.
 2. [Notification Service] Dispatches a notification to the target user:
 - `type = FOLLOW`
 - `message = "@follower_username started following you!"`
 3. [Backend] Returns 200 OK with `{ following: true }`.
 4. [Frontend] Updates button text to “Following”.
-

11.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Return 401, stop
Self-follow?	Yes	Return 400 Bad Request, stop
Record exists?	Yes	Delete follow record, return <code>following: false</code>

Decision	Path/Condition	Outcome
Record exists?	No	Create follow record, notify target, return following: true

11.5 Key Implementation Details

- **Database-Level Integrity:** The unique composite index on (`follower_id`, `following_id`) ensures that a follower can never create multiple records for the same target artist in the database.
- **Asymmetric Notification Rules:** While a new follow event triggers an instant notification for the artist, an unfollow event silently deletes the record without sending any alerts, protecting user privacy.

12 Activity Diagram - Google OAuth

UML Type: Activity Diagram Category: Auth Source: docs/diagrams/activity/auth/activity_google_oauth

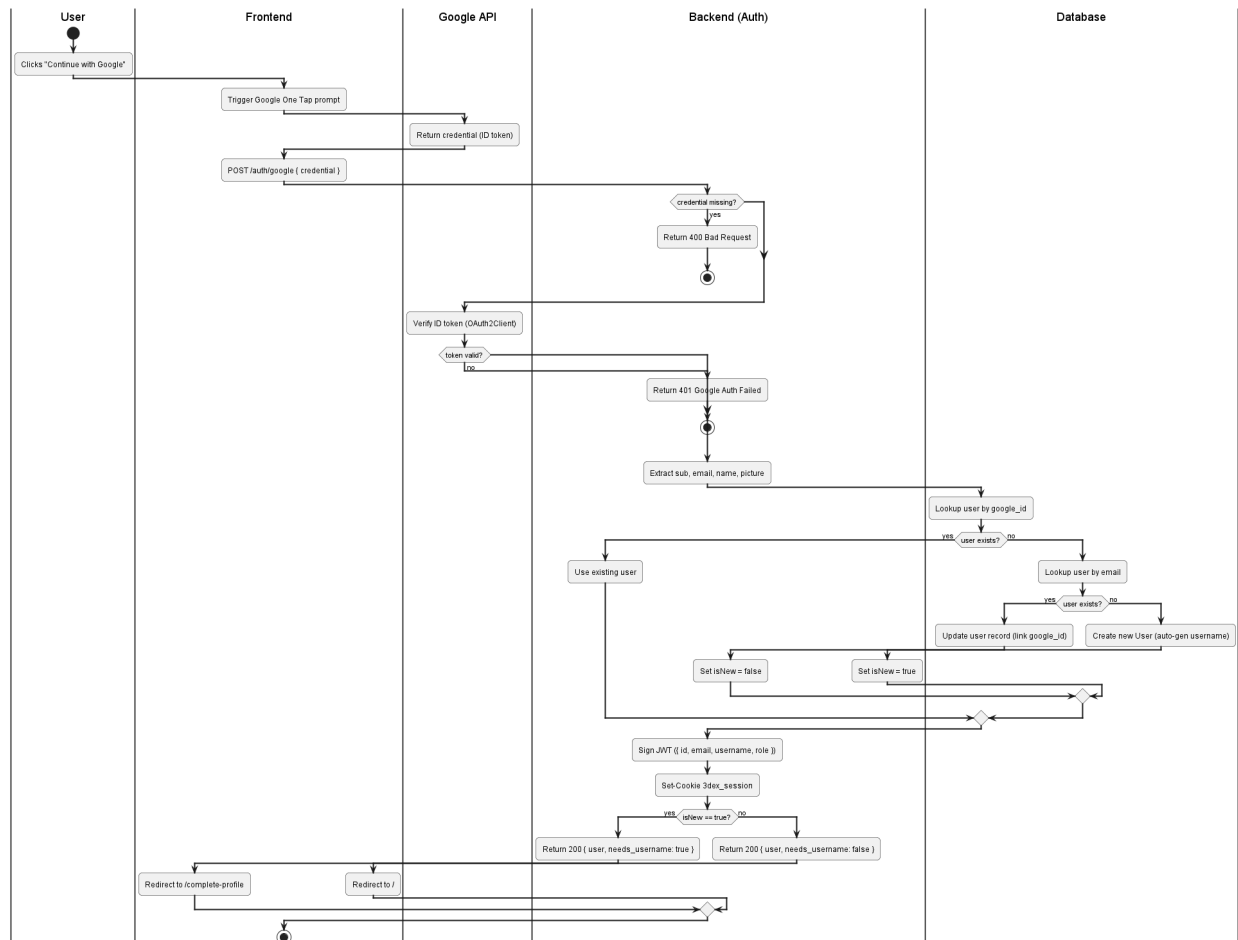


Figure 12: Activity Diagram - Google OAuth

12.1 Overview

This activity diagram maps the full process flow for Google OAuth authentication through swim lanes. It is more detailed than the sequence diagram in that it explicitly shows the three-way user-lookup branch.

12.2 Swim Lanes

Lane	Responsibility
User	Initiates the OAuth flow
Frontend	Manages the Google prompt and response
Backend	Verifies token and handles all three lookup paths
Database	Stores and returns user data

12.3 Process Flow

1. [User] Clicks “Continue with Google”.
2. [Frontend] Triggers Google One Tap or OAuth redirect.
3. [Google Auth] Returns a credential (ID token).
4. [Frontend] Sends POST /auth/google { credential }.
5. [Backend] Validates that credential is present.
 - If missing: returns 400.
6. [Backend] Calls Google’s tokeninfo to verify the token.
 - If verification fails: returns 401.
7. [Backend] Extracts { sub, email, name, picture } from payload.
8. [Database] Attempts to find user by google_id = sub.

Three-way decision:

Path A - Google user found: 9A. [Backend] Uses the existing user directly.

Path B - No Google match, existing email user: 9B. [Database] Finds user by email. 10B. [Database] Updates that user’s record: google_id = sub. 11B. [Backend] Sets isNew = false.

Path C - Completely new user: 9C. [Database] Creates new user: { google_id: sub, email, role: CUSTOMER, account_status: APPROVED }. 10C. [Backend] Sets isNew = true.

Rejoin:

10. [Backend] Signs JWT, sets 3dex_session cookie.
 11. [Backend] Returns 200.
 12. [Frontend] Checks needs_username:
 - If true (isNew): redirects to /complete-profile.
 - Otherwise: redirects to home.
-

12.4 Decision Points Summary

Decision	Path
Missing credential	400, stop
Token verification fails	401, stop
User found by google_id?	Path A

Decision	Path
No google match, user found by email?	Path B
Neither match	Path C
needs_username?	Redirect to complete-profile

12.5 Notes

- Path B (account linking) silently links the Google account to an existing local account without any user action. This is transparent to the user.
- The `isNew` flag controls whether the user is forced to set a username before using the platform.

13 Activity Diagram - Local Login

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_local_login.p

13.1 Overview

This activity diagram models the process flow of local (email + password) login as a series of steps and decisions distributed across the Admin, Frontend, Backend, and Database swim lanes.

13.2 Swim Lanes

Lane	Responsibility
User	Interacts with the login form
Frontend	Validates input client-side and sends the request
Backend	Executes authentication logic
Database	Stores and returns user data

13.3 Process Flow

1. [User] Opens the login form.
2. [Frontend] User submits email and password.
3. [Frontend] Client-side validation: checks fields are not empty.
 - If empty: shows a field required error. Flow ends.
4. [Frontend] Sends `POST /auth/login { email, password }`.
5. [Backend] Receives the request; runs field presence validation again.
 - If missing fields: returns 400.
6. [Database] Looks up user by email.
 - If not found: Backend returns 401 Invalid credentials.
7. [Backend] Calls `bcrypt.compare(input, hash)`.
 - If mismatch: returns 401 Invalid credentials.
8. [Database] Updates `last_login_at`.
9. [Backend] Signs JWT, sets `3dex_session` cookie.
10. [Backend] Returns 200 with user object.
11. [Frontend] Stores user in `AuthContext`, redirects to home.

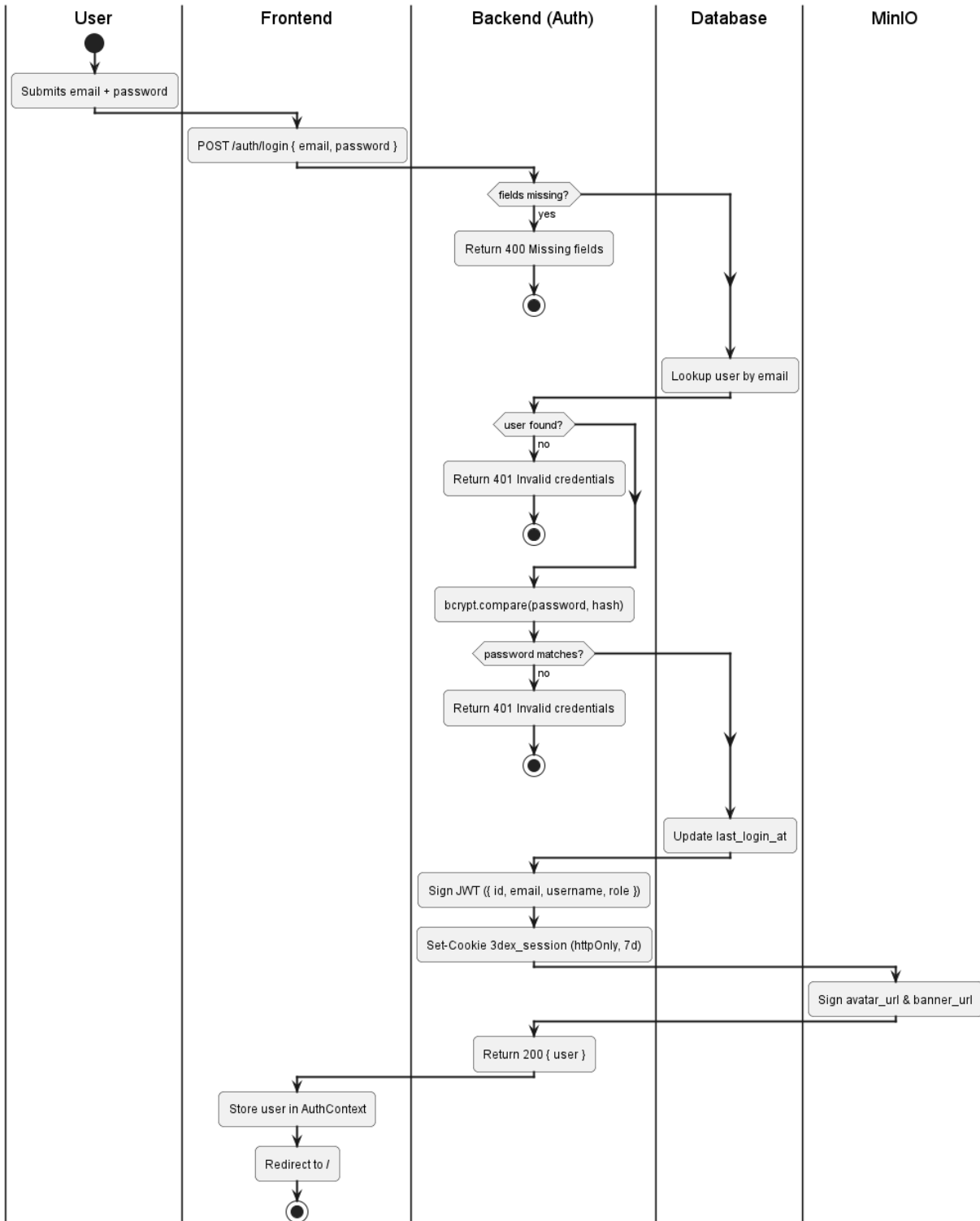


Figure 13: Activity Diagram - Local Login

13.4 Decision Points Summary

Decision	Yes Path	No Path
Fields empty?	Show error, stop	Continue
User found?	Continue	Return 401
Password match?	Continue	Return 401

13.5 Relationship to Sequence Diagram

This diagram reinforces the same flow shown in `sequence_local_login.puml` but uses swim lanes to emphasize which system component is responsible for each step, making the responsibility boundaries clearer for implementation purposes.

13.6 Notes

- The double validation (client-side and server-side) is intentional. Client-side validation gives immediate feedback; server-side validation is the authoritative gate.
- The `bcrypt.compare` step happens only after confirming the user exists, preventing unnecessary computation on non-existent accounts.

14 Activity Diagram - Model Delete

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_model_d

14.1 Overview

This activity diagram describes the procedural flow and access gates when a model is deleted from the platform. It tracks two distinct execution paths: an Owner (Artist) deletion request and an Admin deletion request, detailing authorization checks and database cascading behavior.

14.2 Swim Lanes

Lane	Responsibility
Artist or Admin	Initiates the deletion flow and inputs justifications
Frontend	Captures action and payload, sends request
Backend	Executes business rules, validates roles and ownership
Database	Performs lookup, updates logs, and executes cascading deletions

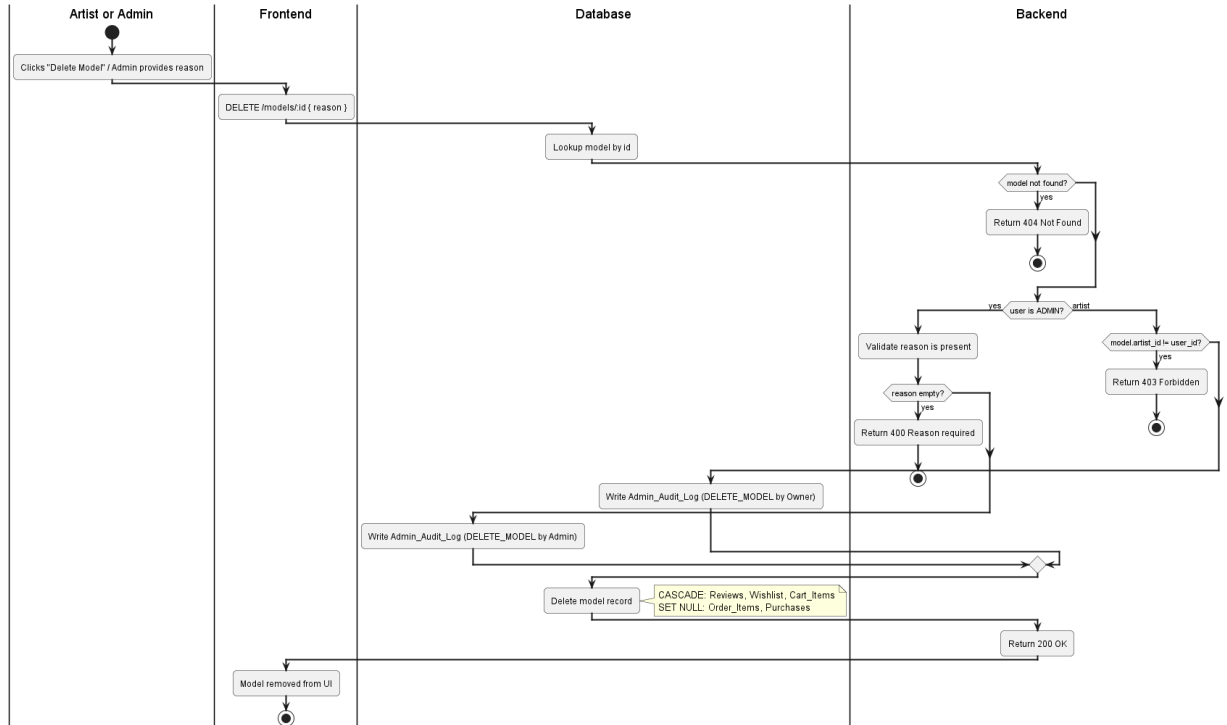


Figure 14: Activity Diagram - Model Delete

14.3 Process Flow

1. **[Artist or Admin]** Triggers the “Delete Model” action in the UI. If the actor is an Admin, they must also provide a deletion reason.
2. **[Frontend]** Dispatches `DELETE /models/:id { reason }`.
3. **[Database]** Looks up the target model by ID.
4. **[Backend]** Checks if the model exists.
 - If not found: returns 404. Flow ends.
5. **[Backend]** Checks if the user is an Admin.

Decision Branch: User is Admin vs. User is Artist (Owner)

- **Path A: User is Admin**
 1. Backend checks if the reason is present and not empty.
 - If reason is empty: returns 400. Flow ends.
 2. Database writes an `Admin_Audit_Log` entry for `DELETE_MODEL` initiated by Admin.
- **Path B: User is Artist**
 1. Backend verifies `model.artist_id == user_id`.
 - If not equal (unauthorized): returns 403. Flow ends.
 2. Database writes an `Admin_Audit_Log` entry for `DELETE_MODEL` initiated by Owner.

Rejoin and Final Execution:

6. **[Database]** Deletes the model record from the database.
7. **[Database]** Triggers database-level cascade actions:
 - **CASCADE (Deleted):** Review, Wishlist, Cart_Items, Collection_Item records referencing the model.
 - **SET NULL:** Order_Items and Purchases fields pointing to the model are cleared (nullified) to preserve financial history.
8. **[Backend]** Returns 200 OK.

9. **[Frontend]** Removes the model from the current UI context.

14.4 Decision Points Summary

Decision	Path/Condition	Outcome
Model exists?	No	Return 404, stop
Caller role?	Admin	Check reason; if empty, return 400. Write Admin Audit Log
Caller role?	Artist	Check ownership; if unauthorized, return 403. Write Owner Audit Log
Constraint Check	Cascade	Delete dependent lists; nullify purchases

14.5 Key Implementation Details

- **Cascade Rules:** The prisma schema designates `onDelete: Cascade` for temporary user lists (Cart, Wishlist) and rating systems (Reviews). However, for completed transactions and operational records (`Purchases`, `Order_Items`), it uses `onDelete: SetNull` so transaction lists remain structurally intact.
- **Audit Logs:** The database logs owner deletions too, providing a comprehensive log of all content removals.

15 Activity Diagram - Model Download

UML Type: Activity Diagram Category: Catalog Source: docs/diagrams/activity/catalog/activity_model_d

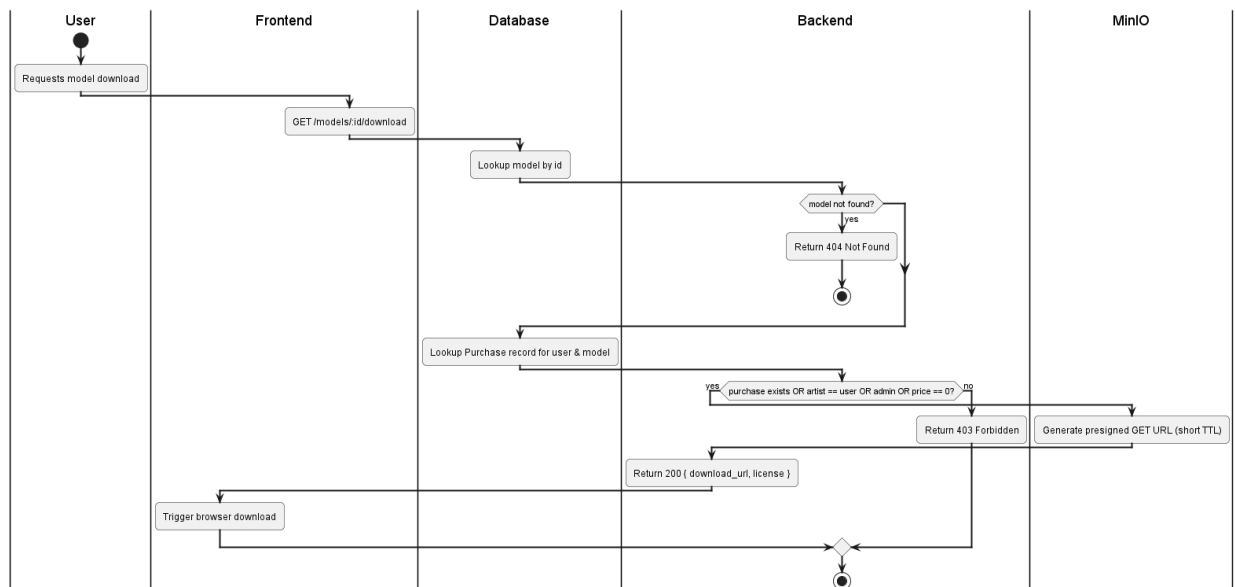


Figure 15: Activity Diagram - Model Download

15.1 Overview

This activity diagram describes the validation sequence and delivery mechanisms of the model download process. It focuses on the server-side checks that must pass before a secure S3 presigned URL is served to the client browser.

15.2 Swim Lanes

Lane	Responsibility
User	Requests model download
Frontend	Issues download API call and handles browser trigger
Backend	Runs access rules and generates short-lived URLs
Database	Performs model lookup and purchase verification
MinIO	Issues timed presigned download URL

15.3 Process Flow

1. [User] Clicks the “Download” trigger on the model page.
2. [Frontend] Sends GET `/models/:id/download` with user credentials.
3. [Database] Looks up the model by its unique ID.
4. [Backend] Checks if the model exists.
 - If not found: returns 404. Flow ends.
5. [Database] Queries for a valid **Purchase** record matching the requesting `user_id` and target `model_id`.
6. [Backend] Evaluates download eligibility based on the following OR rules:
 - Does a validated **Purchase** record exist in the database?
 - Is the current `user_id` equal to the model’s `artist_id` (owner check)?
 - Does the user possess the **ADMIN** role?
 - Is the model’s listing price equal to 0 (free asset)?
7. [Backend] Branching:
 - **No (none of the above are true):** Returns 403 Forbidden. Flow ends.
 - **Yes (at least one is true):**
 1. [MinIO] Backend requests MinIO to generate a short-lived presigned GET URL (short TTL).
 2. [Backend] Returns 200 OK along with { `download_url`, `license` }.
 3. [Frontend] Intercepts the payload and triggers a native browser download pointing to the generated presigned S3 URL.

15.4 Decision Points Summary

Decision	Path/Condition	Outcome
Model exists?	No	Return 404, stop
Eligible? (Purchase, Owner, Admin, or Free)	Yes	Call MinIO, generate presigned URL, return 200
Eligible?	No	Return 403 Forbidden, stop

15.5 Key Implementation Details

- **Security Gatekeeping:** Access verification is performed strictly on the Backend prior to generating any S3 URLs. Simply having access to the client model page does not bypass this gate.
- **Short TTL URLs:** The S3 URL uses a very short Time-To-Live (e.g., 5 to 15 minutes) to ensure that even if the link is intercepted or shared, it becomes inactive shortly after generation.
- **Free Models:** The check `price == 0` allows any logged-in user to download free assets immediately, bypassing the need for a `Purchase` record.

16 Activity Diagram - Model Review

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_model_r

16.1 Overview

This activity diagram describes the validation sequence and denormalization updates when a customer submits a review (rating and optional text comment) for a 3D model.

16.2 Swim Lanes

Lane	Responsibility
Customer	Submits the review details
Frontend	Form validation and UI updates
Backend	Runs business logic, verifies purchase, and triggers updates
Database	Persists review, checks constraints, and recalculates averages

16.3 Process Flow

1. **[Customer]** Fills out the review form (rating between 1-5 and comment text) and clicks submit.
 2. **[Frontend]** Dispatches `POST /models/:id/reviews` with the request body.
 3. **[Backend]** Validates that `rating` is an integer between 1 and 5.
 - If invalid: returns 400 Bad Request. Flow ends.
 4. **[Database]** Looks up the `Purchase` record for the user and model.
 5. **[Backend]** Checks if the model was purchased.
 - If not purchased: returns 403 Forbidden. Flow ends.
 6. **[Database]** Attempts to insert a new `Review` record.
 7. **[Database]** Checks for duplicate constraint violations (Prisma unique constraint P2002 on (`user_id`, `model_id`)).
 - If a review already exists: Backend returns 409 Conflict. Flow ends.
 8. **[Database]** Executes aggregation query: computes new count and average rating for the model.
 9. **[Database]** Updates the `Model` record with the updated `avg_rating` and `review_count`.
 10. **[Backend]** Returns 201 Created.
 11. **[Frontend]** Appends the review to the model detail page review list.
-

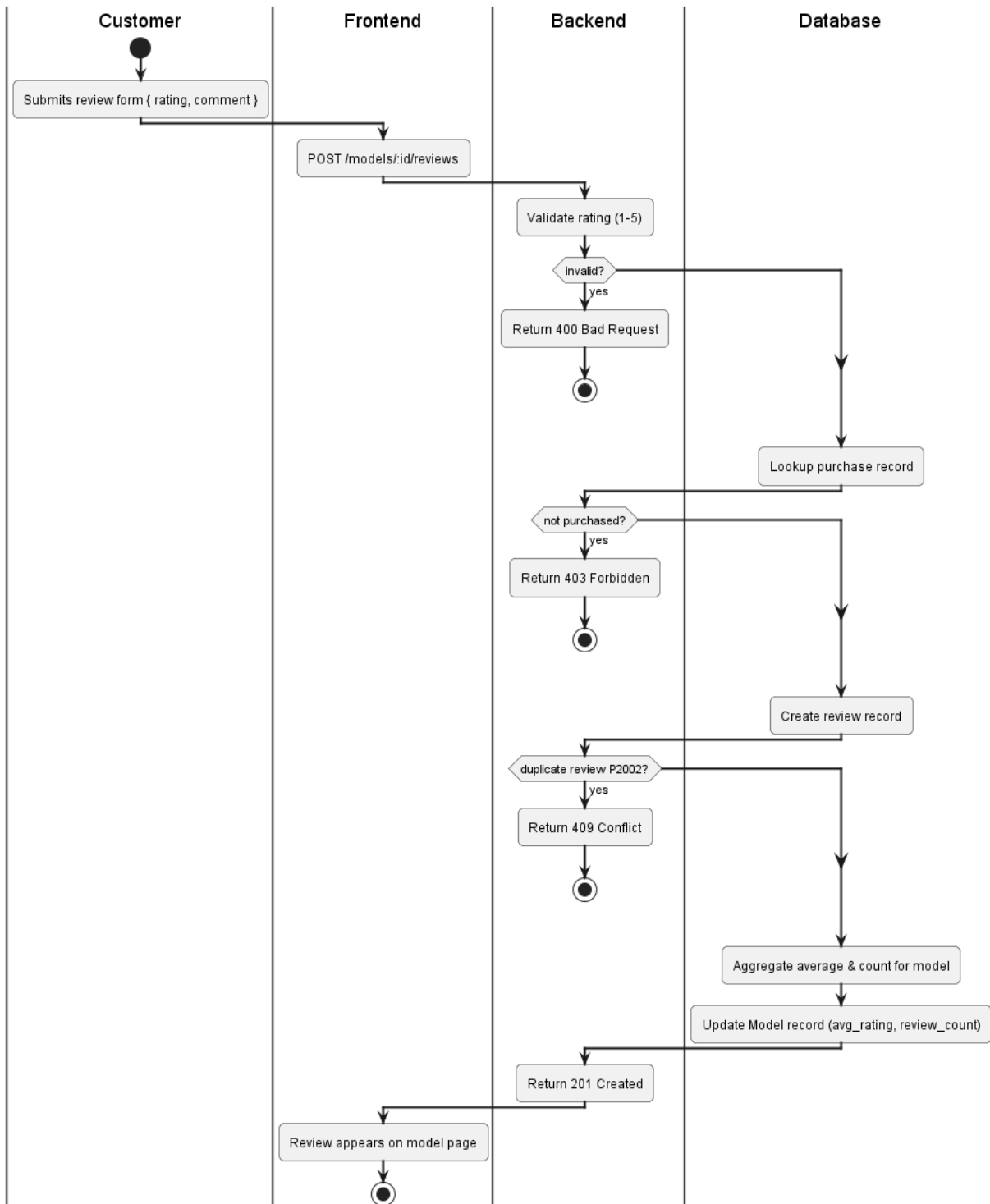


Figure 16: Activity Diagram - Model Review

16.4 Decision Points Summary

Decision	Path/Condition	Outcome
Rating is valid (1-5)?	No	Return 400, stop
Purchased by user?	No	Return 403, stop
Already reviewed?	Yes (P2002)	Return 409, stop

16.5 Key Implementation Details

- **Denormalization Strategy:** To avoid executing heavy database aggregations (AVG and COUNT) on every catalog listing query, the system keeps denormalized `avg_rating` and `review_count` fields directly on the `Model` record. These are updated atomically whenever a new review is successfully persisted.
- **Purchase Restriction:** Verified purchase validation is mandatory. Users cannot review models they do not own.

17 Activity Diagram - Model Upload

UML Type: Activity Diagram Category: Catalog Source: docs/diagrams/activity/catalog/activity_model_upload

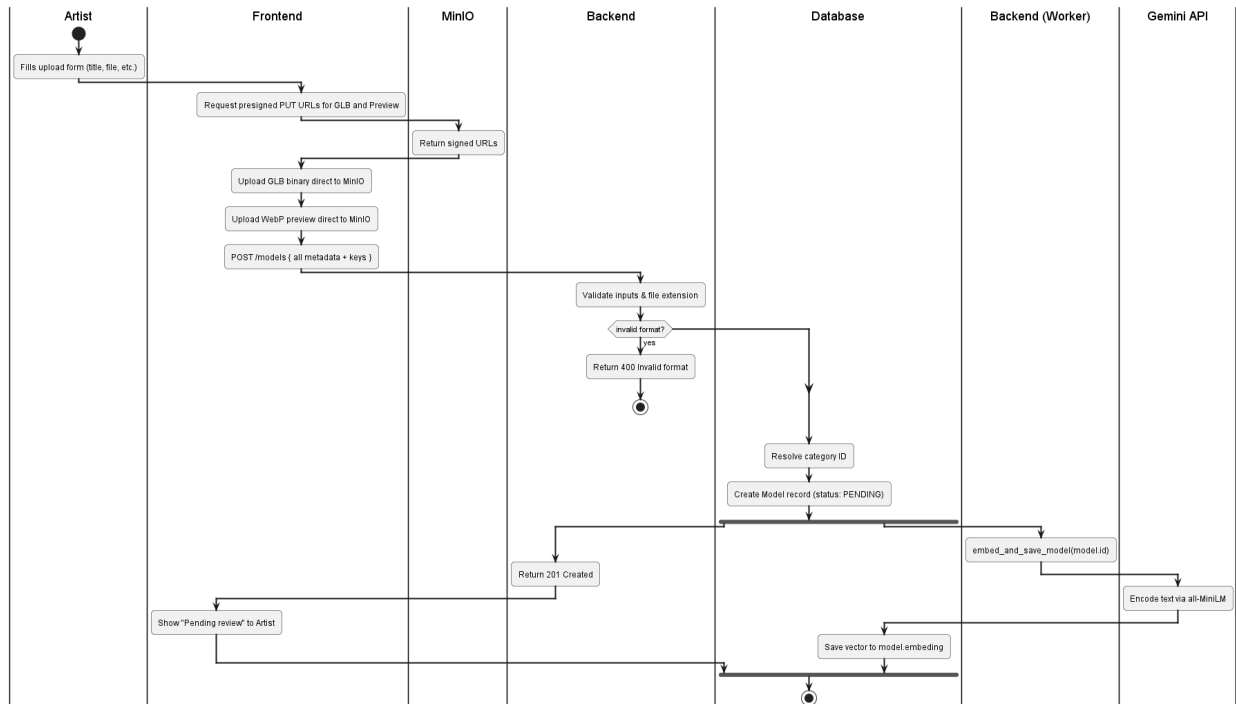


Figure 17: Activity Diagram - Model Upload

17.1 Overview

This activity diagram shows the model upload process with swim lanes, emphasizing the parallel fork between the immediate 201 Created response and the asynchronous background embedding generation.

17.2 Swim Lanes

Lane	Responsibility
Artist	Fills and submits the upload form
Frontend	Requests presigned URLs and sends file data
MinIO	S3-compatible storage for file upload
Backend	Validates metadata and creates the model record
Backend (Worker)	Asynchronous embedding generation
Gemini API	Text embedding model endpoint
Database	Persists model record and vector embedding

17.3 Process Flow

1. **[Artist]** Fills upload form (title, file, tags, category, etc.).
2. **[Frontend]** Requests presigned PUT URLs for GLB and WebP preview files.
3. **[MinIO]** Returns signed URLs.
4. **[Frontend]** Uploads GLB binary directly to MinIO.
5. **[Frontend]** Uploads WebP preview directly to MinIO.
6. **[Frontend]** Sends `POST /models { all metadata + S3 keys }`.
7. **[Backend]** Validates inputs and checks file extension.
 - If invalid format: returns 400. Flow ends.
8. **[Database]** Resolves `category_id` from slug.
9. **[Database]** Creates `Model` record with `status: PENDING`.

Fork (parallel):

Main path: 10A. **[Backend]** Returns 201 Created. 11A. **[Frontend]** Shows “Pending review” to Artist.

Background path: 10B. **[Backend (Worker)]** Calls `embed_and_save_model(model.id)`. 11B. **[Gemini API]** Encodes model title/description text via `all-MiniLM`. 12B. **[Database]** Saves 384-dim vector to `model.embedding`.

17.4 Decision Points Summary

Decision	Yes Path	No Path
File format valid (glb/gltf)?	Continue	Return 400, stop

17.5 Notes

- The parallel fork after model creation reflects the fire-and-forget nature of embedding generation. The user does not wait for the embedding to complete.
- The diagram labels the embedding model as “all-MiniLM” which refers to `all-MiniLM-L6-v2`. It is noted in the diagram as executed via Gemini API context, though in practice the embedding is done using a sentence-transformer library, not the Gemini Flash text generation API.

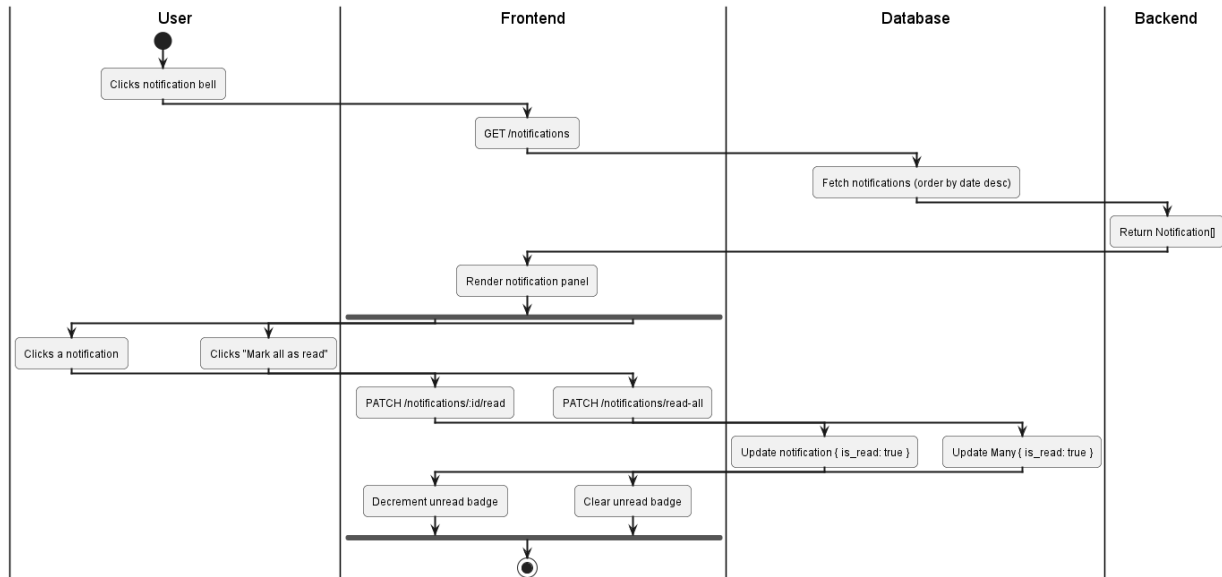


Figure 18: Activity Diagram - Notifications

18 Activity Diagram - Notifications

UML Type: Activity Diagram **Category:** Notifications **Source:** docs/diagrams/activity/notifications/activi

18.1 Overview

This activity diagram describes the notification pull and mark-as-read workflow. It highlights the operations executed when loading lists, updating single notification records, and clearing all notifications at once.

18.2 Swim Lanes

Lane	Responsibility
User	Interacts with notification panel trigger and item links
Frontend	Issues API calls and updates local state counters
Backend	Runs session validation and constructs updates
Database	Queries and updates the <code>Notification</code> table

18.3 Process Flow

1. [User] Clicks the notification bell icon.
2. [Frontend] Dispatches `GET /notifications` to load history.
3. [Database] Queries `Notification` table: `findMany({ user_id, orderBy: created_at desc })`.
4. [Backend] Returns 200 OK along with notifications array.
5. [Frontend] Renders the list and updates the global unread count badge.

User Interaction Fork (mutually exclusive choices):

- **Choice A: Click Single Notification**
 1. [User] Clicks a single notification item in the popover.
 2. [Frontend] Dispatches PATCH /notifications/:id/read.
 3. [Database] Updates Notification record: `is_read = true`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Decrements the navbar unread count by 1 and navigates to the notification's destination path.
- **Choice B: Click "Mark All as Read"**
 1. [User] Clicks "Mark all as read" button.
 2. [Frontend] Dispatches POST /notifications/read-all.
 3. [Database] Updates all matching records: `updateMany({ user_id, is_read: false }, { is_read: true })`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Sets the unread count badge to 0.

18.4 Decision Points Summary

Decision	Path/Condition	Outcome
Click action?	Single	Send patch, update single item, decrement badge count, navigate
Click action?	Mark All	Send read-all post, update all records to read, zero badge count

18.5 Key Implementation Details

- **Deep Linking:** The Notification model includes a JSON data column. This holds contextual details (e.g., `{ model_id: "...", post_id: "..." }`), which the frontend reads when a notification is clicked to determine where to redirect the user.
- **Atomic Read-All Updates:** The `updateMany` operation on the Database runs as a single SQL update command, ensuring fast execution even if the user has many unread records.

19 Activity Diagram - Order Cancel

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_order

19.1 Overview

This activity diagram describes the order cancellation flow. It covers how a user manually cancels a pending order (or how it expires), and how the system communicates this state change to the Midtrans payment gateway to void the transaction.

19.2 Swim Lanes

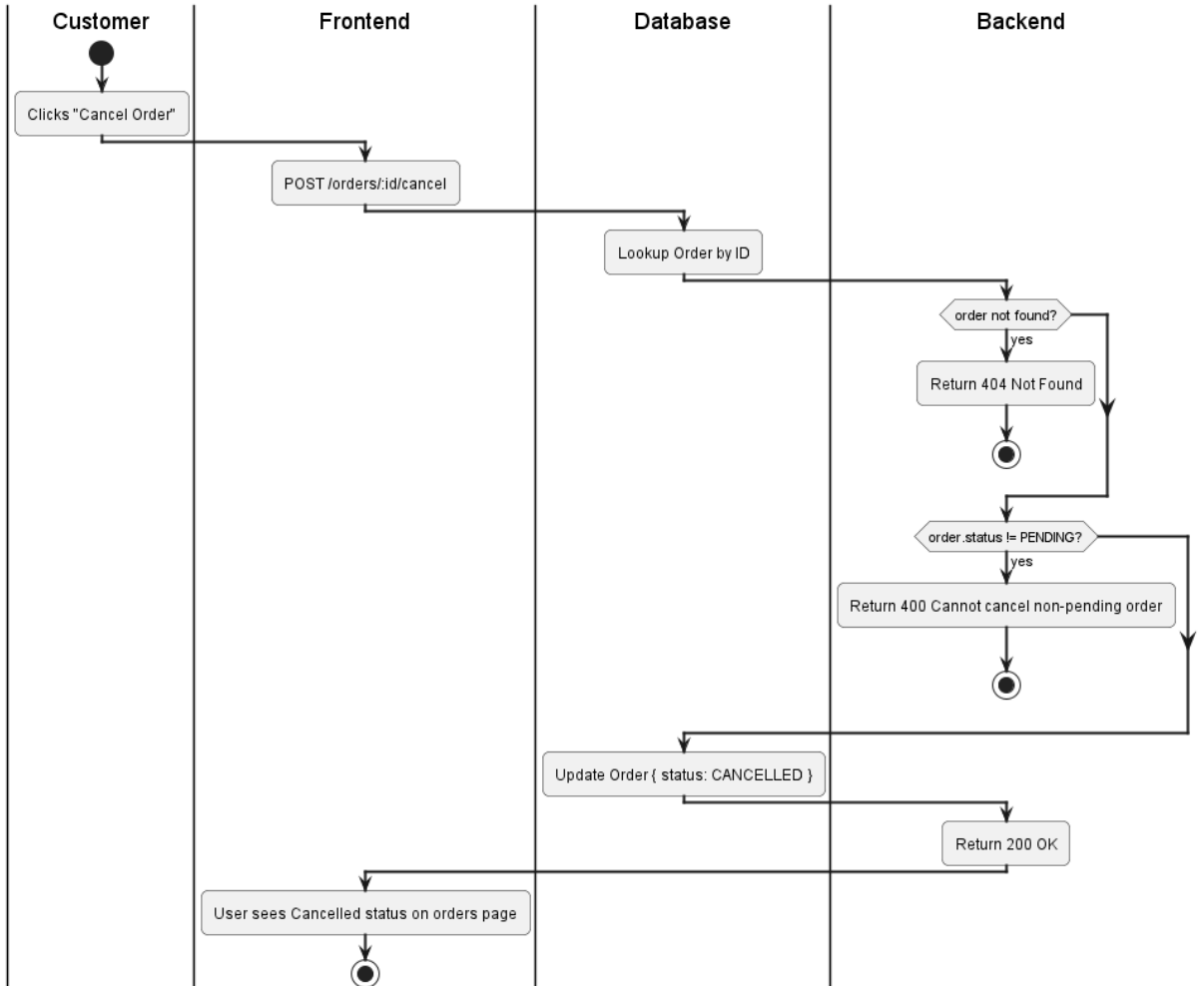


Figure 19: Activity Diagram - Order Cancel

Lane	Responsibility
Customer or Cron System	Initiates cancellation or triggers expiry
Frontend	Sends cancellation request
Backend	Runs checks, updates order, and calls Midtrans
Database	Updates Order status in PostgreSQL
Midtrans API	Voids or cancels transaction on the gateway side

19.3 Process Flow

1. **[Customer or Cron System]** Clicks “Cancel Order” on the order details page, or the background cron system finds an order that has exceeded its payment window.
2. **[Frontend]** Dispatches `POST /orders/:id/cancel`.
3. **[Database]** Looks up the target `Order` by ID.
4. **[Backend]** Checks if the order exists.
 - If not found: returns 404. Flow ends.
5. **[Backend]** Verifies the order status is currently `PENDING`.
 - If status is already `PAID` or `CANCELLED`: returns 400 Bad Request. Flow ends.
6. **[Backend]** Calls the **[Midtrans API]** `/cancel` endpoint for this transaction to cancel the payment token.
7. **[Database]** Updates the order: `status = CANCELLED`.
8. **[Backend]** Returns 200 OK.
9. **[Frontend]** Updates order view to show “Cancelled”.

19.4 Decision Points Summary

Decision	Path/Condition	Outcome
Order exists?	No	Return 404, stop
Status is <code>PENDING</code> ?	No	Return 400 Bad Request, stop
Midtrans cancel call?	-	Informs gateway to close Snap token

19.5 Key Implementation Details

- **Gateway Sync:** Simply marking an order as `CANCELLED` in our local database is not enough. We must explicitly call Midtrans to cancel/void the transaction so the user cannot use the previous Snap token or redirect URL to pay.
- **Auto-Expiry:** In addition to manual cancellations, a background worker runs periodically to transition orders that are still pending after 24 hours to `CANCELLED` status, triggering the same Midtrans cancellation endpoint.

20 Activity Diagram - Payment Webhook

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_paym

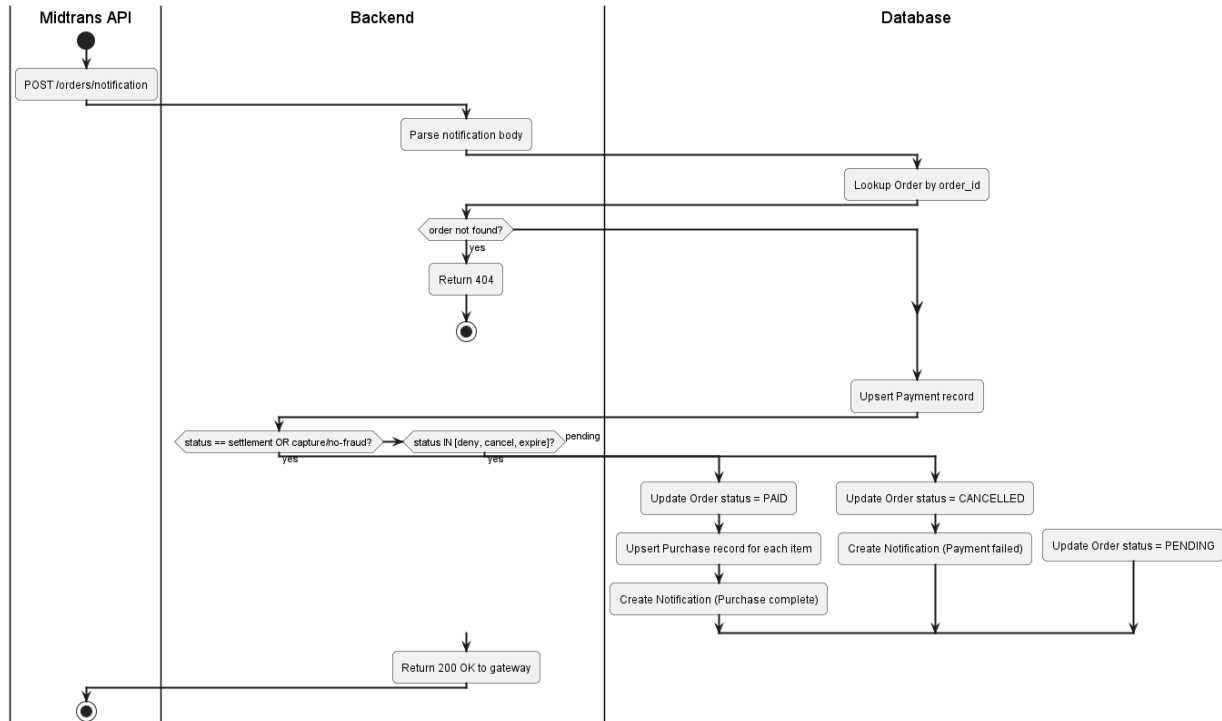


Figure 20: Activity Diagram - Payment Webhook

20.1 Overview

This activity diagram describes the system's asynchronous response to the Midtrans payment notification webhook. It details how the transaction status is read, how payments are upserted for idempotency, and how the user's purchased assets are unlocked.

20.2 Swim Lanes

Lane	Responsibility
Midtrans API	Dispatches POST notification webhook
Backend	Parses notifications, routes logic, and verifies signatures
Database	Updates Orders, creates Purchases, and logs transactions
Notification Service	Sends alerts to the user's notification list

20.3 Process Flow

1. [Midtrans API] Sends POST `/orders/notification` with transaction details.
2. [Backend] Verifies the webhook signature using the shared merchant server key.
 - If signature is invalid: returns 401 Unauthorized. Flow ends.
3. [Database] Looks up the target `Order` by the provided `order_id`.
 - If order not found: Backend returns 404. Flow ends.

4. **[Database]** Upserts a **Payment** record using the `transaction_id` as the key. This ensures the webhook handling is idempotent.
5. **[Backend]** Evaluates the transaction status parameter.

Conditional Path Branching:

- **Path A: Settlement or Capture (Success)**
 - Conditions: `transaction_status == settlement` OR (`transaction_status == capture` and `fraud_status != deny`).
 - 1. **[Database]** Updates `Order.status = PAID`.
 - 2. **[Database]** Creates a **Purchase** record for each `Order_Item` in the order, unlocking the model for download.
 - 3. **[Notification Service]** Dispatches a notification to the customer.
- **Path B: Deny, Cancel, or Expire (Failure)**
 - Conditions: `transaction_status` is in `[deny, cancel, expire]`.
 - 1. **[Database]** Updates `Order.status = CANCELLED`.
 - 2. **[Notification Service]** Dispatches a notification to the customer indicating failure.
- **Path C: Pending**
 - Conditions: `transaction_status == pending`.
 - 1. **[Database]** Leaves `Order.status = PENDING`.

Rejoin:

6. **[Backend]** Returns 200 OK to Midtrans.

20.4 Decision Points Summary

Decision	Path/Condition	Outcome
Signature Valid?	No	Return 401 Unauthorized, stop
Status Settlement/Capture?	Yes	Order = PAID, Create Purchases, notify user
Status Deny/Cancel/Expire?	Yes	Order = CANCELLED, notify user
Status Pending?	Yes	Keep Order = PENDING

20.5 Key Implementation Details

- **Idempotency via Upsert:** Since Midtrans may retry webhooks if network glitches occur, the **Payment** upsert avoids duplicate payment processing. If a payment record with the same transaction ID already exists, its details are updated but duplicate **Purchase** records are not created.
- **Secure Webhook Verification:** The system computes a SHA-512 hash using `order_id`, `status_code`, `gross_amount`, and the server key, comparing it to the webhook's `signature_key` to prevent fraud.

21 Activity Diagram - Post Create

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_post_creat

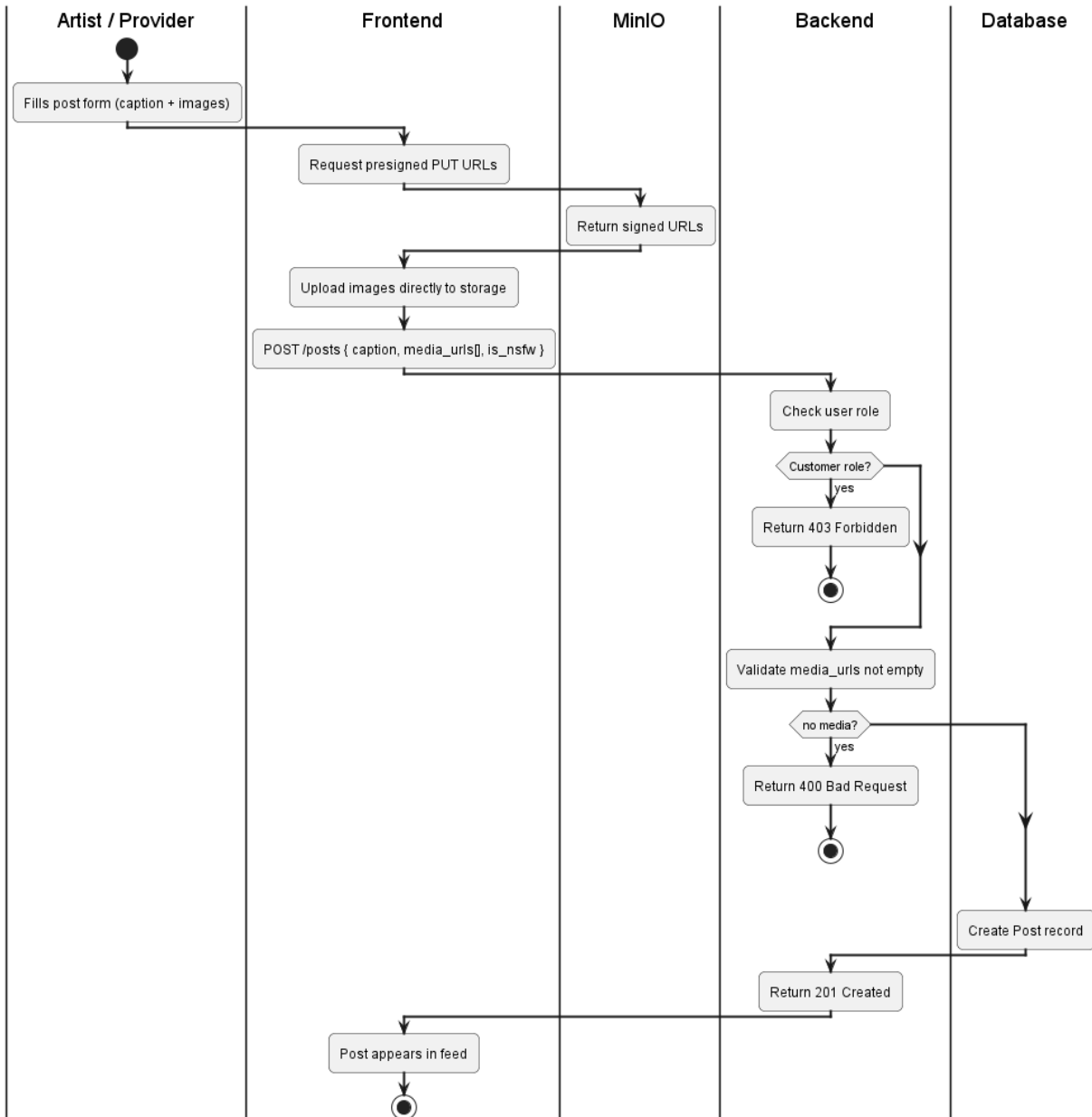


Figure 21: Activity Diagram - Post Create

21.1 Overview

This activity diagram describes the process of creating a new community post. It outlines the role-based validation checks (only Artists and Providers are allowed to post) and S3 file attachment flows.

21.2 Swim Lanes

Lane	Responsibility
Artist or Provider Frontend	Fills out caption and uploads images Requests presigned URLs, uploads to MinIO, and sends metadata
MinIO Backend	S3-compatible storage backend Runs role checks, validates input, and persists the post
Database	Inserts the new Post record

21.3 Process Flow

1. **[Artist or Provider]** Fills out the caption, sets NSFW toggles, and selects images.
2. **[Frontend]** Requests presigned PUT URLs from the backend for each image.
3. **[MinIO]** Returns signed S3 upload URLs.
4. **[Frontend]** Uploads raw image binaries directly to MinIO using the presigned URLs.
5. **[Frontend]** Dispatches `POST /posts` with `{ caption, media_urls: [keys], is_nsfw }`.
6. **[Backend]** Verifies user credentials and role.
 - If user role is `CUSTOMER` or `ADMIN`: returns 403 Forbidden (“Only Artists and Providers can create community posts”). Flow ends.
7. **[Backend]** Validates the request payload:
 - Verifies the caption length and formats.
 - Verifies `media_urls` is not empty (community posts must contain at least one image/media attachment).
 - If invalid: returns 400 Bad Request. Flow ends.
8. **[Database]** Inserts the new `Post` record containing the media paths and initial zeroed statistics (`like_count = 0, comment_count = 0`).
9. **[Backend]** Returns 201 Created along with the new post details.
10. **[Frontend]** Appends the post to the top of the community feed.

21.4 Decision Points Summary

Decision	Path/Condition	Outcome
Role is Artist or Provider?	No	Return 403 Forbidden, stop
Media attachments exist?	No	Return 400 Bad Request, stop

21.5 Key Implementation Details

- **Role Constraints:** The `Post` table represents promotional or portfolio updates. To maintain high-quality feed content, only approved creators (`ARTIST` or `PROVIDER`) are authorized to upload posts. Regular users (`CUSTOMER`) can interact through likes and comments but cannot post directly.
- **Direct S3 Uploads:** Similar to model uploads, image binaries are uploaded directly to MinIO storage from the frontend to keep the application server highly responsive.

22 Activity Diagram - Post Delete

UML Type: Activity Diagram Category: Social Source: docs/diagrams/activity/social/activity_post_delete

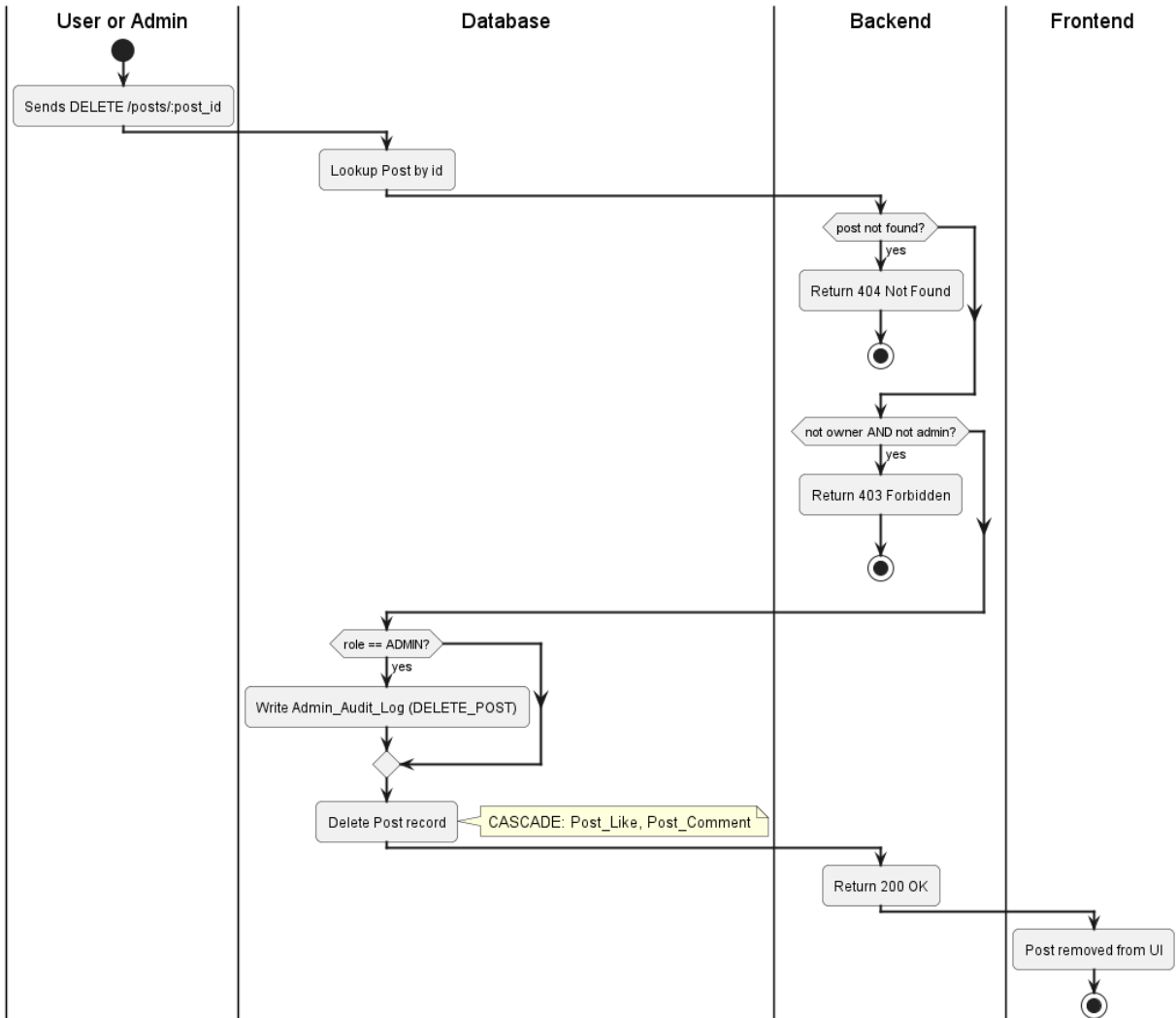


Figure 22: Activity Diagram - Post Delete

22.1 Overview

This activity diagram describes the community post deletion process. It highlights the security checks verifying either author ownership or admin moderator status before deleting post and cascade-related data.

22.2 Swim Lanes

Lane	Responsibility
Author or Admin	Triggers deletion
Frontend	Dispatches delete request
Backend	Verifies session, matches ownership, and authorizes deletion
Database	Performs lookups and handles cascade deletions

22.3 Process Flow

1. **[Author or Admin]** Clicks the “Delete Post” button on a feed card.
 2. **[Frontend]** Dispatches `DELETE /posts/:id` with user headers.
 3. **[Database]** Looks up the post by ID.
 4. **[Backend]** Checks if the post exists.
 - If not found: returns 404. Flow ends.
 5. **[Backend]** Evaluates authorization boundaries:
 - Is the user ID equal to `post.user_id` (ownership check)?
 - Does the user possess the `ADMIN` role?
 6. **[Backend]** Branching:
 - **No (neither is true):** Returns 403 Forbidden. Flow ends.
 - **Yes (at least one is true):**
 1. **[Database]** Deletes the `Post` record.
 2. **[Database]** Executes database-level cascades:
 - Deletes associated `Post_Like` records.
 - Deletes associated `Post_Comment` records.
 3. **[Backend]** Returns 200 OK.
 4. **[Frontend]** Removes the post card from the community feed.
-

22.4 Decision Points Summary

Decision	Path/Condition	Outcome
Post exists?	No	Return 404, stop
Authorized? (Owner or Admin)	Yes	Delete post, cascade comments and likes, return 200
Authorized?	No	Return 403 Forbidden, stop

22.5 Key Implementation Details

- **Cascade Settings:** The prisma schema designates cascading deletes for post engagements. Deleting a post automatically wipes clean its dependent likes and comments from PostgreSQL.
- **Admin Moderation Bypass:** The admin bypass allows platform moderators to remove offensive or non-compliant posts immediately without requiring the author’s credentials.

23 Activity Diagram - Post Like

UML Type: Activity Diagram Category: Social Source: docs/diagrams/activity/social/activity_post_like.

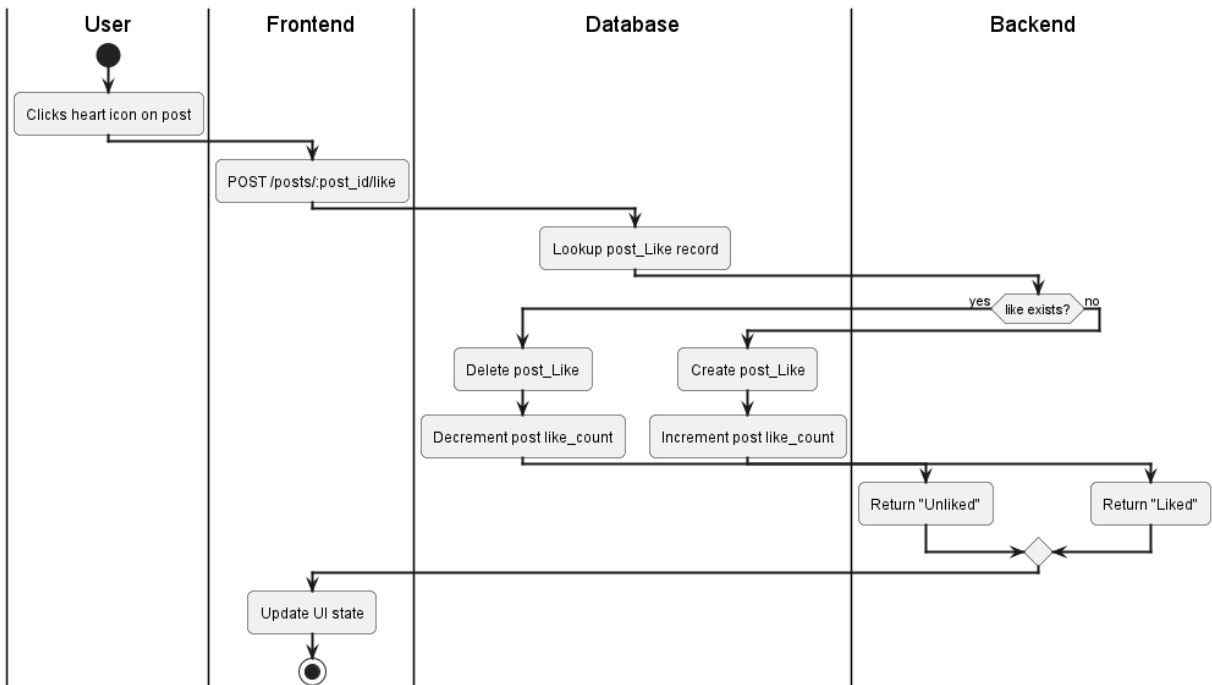


Figure 23: Activity Diagram - Post Like

23.1 Overview

This activity diagram describes the process of toggling a like on a community post. It details the session validation, checks for existing likes, and updates the denormalized like counters.

23.2 Swim Lanes

Lane	Responsibility
User	Clicks the post heart icon
Frontend	Dispatches the toggle API call
Backend	Validates requests and adjusts counter caches
Database	Inserts or deletes like records and updates like counts

23.3 Process Flow

1. [User] Clicks the “Like” (heart) button on a feed card.
2. [Frontend] Dispatches POST /posts/:id/like with user credentials.
3. [Backend] Verifies the user is authenticated.
 - If not: returns 401 Unauthorized. Flow ends.

4. [Database] Queries Post_Like for a record matching (user_id, post_id).
5. [Backend] Branching based on existence check:

Decision Branch: Like Record Exists?

- **Path A: Record Exists (Unlike Post)**
 1. [Database] Deletes the Post_Like record.
 2. [Database] Atomically decrements the like_count field on the target Post record.
 3. [Backend] Returns 200 OK with { liked: false }.
 4. [Frontend] Reverts heart style to outline.
- **Path B: Record Does Not Exist (Like Post)**
 1. [Database] Inserts a new Post_Like record.
 2. [Database] Atomically increments the like_count field on the target Post record.
 3. [Backend] Returns 200 OK with { liked: true }.
 4. [Frontend] Styles heart to filled red.

23.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Return 401, stop
Like record exists?	Yes	Delete like record, decrement post like count, return liked: false
Like record exists?	No	Create like record, increment post like count, return liked: true

23.5 Key Implementation Details

- **Atomic Increments:** To prevent race conditions from concurrent clicks, the backend utilizes Prisma's atomic increment/decrement updates (e.g., like_count: { increment: 1 } or decrement: 1), avoiding out-of-sync counters in the database.
- **Idempotent Unique Constraints:** The unique composite key on (user_id, post_id) inside the Post_Like table guarantees that a user cannot register multiple likes on a single post.

24 Activity Diagram - Print Job Lifecycle

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_print_job_1

24.1 Overview

This activity diagram details the operational flow of print order fulfillment from the provider's side. It covers three major phases: acceptance, shipment tracking registration, and final completion.

24.2 Swim Lanes

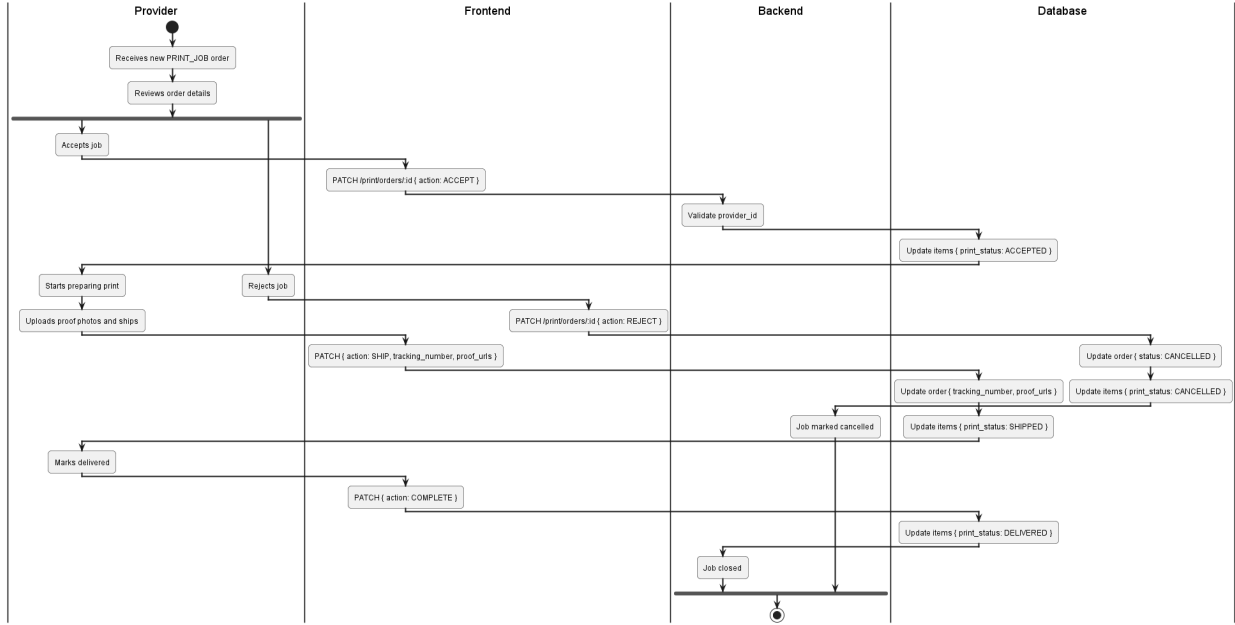


Figure 24: Activity Diagram - Print Job Lifecycle

Lane	Responsibility
Provider	Decides actions, processes raw prints, ships physical package
Frontend	Collects shipping inputs, displays status updates, sends API requests
Backend	Verifies job ownership and routes status transitions
Database	Updates order and item print statuses

24.3 Process Flow

1. [Provider] Opens the print job management dashboard.
2. [Frontend] Dispatches request for the provider's active list (GET /print/jobs).
3. [Database] Queries orders: `findMany({ provider_id, type: PRINT_JOB })`.
4. [Backend] Returns job cards list.

Fulfillment Execution Phases (Sequential Transitions):

24.3.1 Phase 1: Accept Job

1. [Provider] Reviews specifications and clicks "Accept Job".
2. [Frontend] Dispatches POST /print/orders/:id/accept.
3. [Backend] Verifies the requesting user is the assigned provider_id.
 - If mismatch: returns 403. Flow ends.
4. [Database] Updates `Order_Item.print_status = ACCEPTED` for all models in the order.
5. [Backend] Returns 200 OK.

24.3.2 Phase 2: Ship Job

1. [Provider] Finishes 3D printing the model, packages the item, drops it at the courier, and clicks “Ship Job” in the dashboard.
 2. [Frontend] Prompts the provider to input the `tracking_number` and attach proof of shipping photos.
 3. [Provider] Inputs tracking code, uploads photos to S3, and clicks confirm.
 4. [Frontend] Dispatches POST `/print/orders/:id/ship` containing `{ tracking_number, proof_urls }`.
 5. [Database] Updates `Order.tracking_number = tracking_number`, appends `proof_urls` array, and updates `Order_Item.print_status = SHIPPED`.
 6. [Backend] Returns 200 OK.
-

24.3.3 Phase 3: Deliver Job

1. [Provider] Receives delivery notification or clicks “Mark Completed” once delivery is verified.
 2. [Frontend] Dispatches POST `/print/orders/:id/complete`.
 3. [Database] Updates `Order_Item.print_status = DELIVERED`.
 4. [Backend] Returns 200 OK.
 5. [Frontend] Updates order visual state to show complete/closed status.
-

24.4 Decision Points Summary

Decision	Path/Condition	Outcome
Owner check?	Mismatch	Return 403 Forbidden, stop
Action type?	Accept	Update Item print status to ACCEPTED
Action type?	Ship	Update Order tracking parameters; set status to SHIPPED
Action type?	Complete	Update Item print status to DELIVERED

24.5 Key Implementation Details

- **Granular Status Tracking:** Unlike digital purchases where order status transitions directly from PENDING to PAID, physical print jobs require granular item tracking. Thus, the database keeps `print_status` directly on the `Order_Item` table (with states PENDING, ACCEPTED, SHIPPED, DELIVERED).
- **Security Check:** All three endpoints verify that the acting user is the registered `provider_id` of the target order before processing changes.
- **Proof Arrays:** The `proof_urls` are stored as an array of S3 strings in the database, showing photos of the printed model before shipment to protect providers against disputes.

25 Activity Diagram - Print Order

UML Type: Activity Diagram **Category:** Print **Source:** docs/diagrams/activity/print/activity_print_order

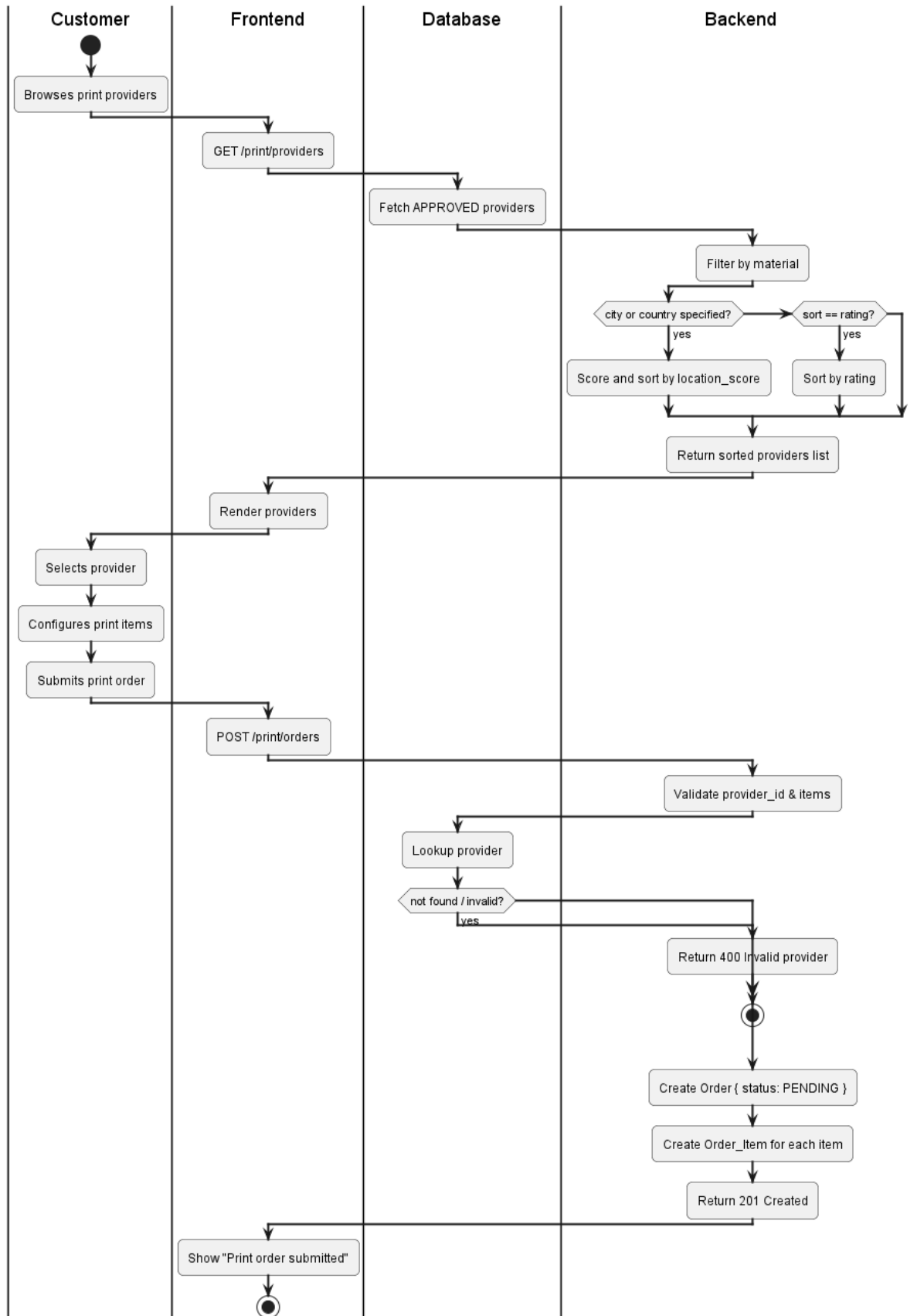


Figure 25: Activity Diagram - Print Order

25.1 Overview

This activity diagram describes the physical 3D print order creation process. It maps how customers browse localized print providers by compatibility parameters and how the order is registered in the database for the provider's queue.

25.2 Swim Lanes

Lane	Responsibility
Customer	Browses providers and configures print jobs
Frontend	Handles filters, queries, and checkout assembly
Backend	Executes routing logic, checks credentials, and processes orders
Database	Performs lookups and creates the Print Job Order record

25.3 Process Flow

1. **[Customer]** Initiates the “Print this model” wizard from a model detail page.
2. **[Frontend]** Dispatches query `GET /print/providers?material=PLA&city=Bandung`.
3. **[Database]** Queries `User` table for records where `role == PROVIDER` and `account_status == APPROVED`.
4. **[Backend]** Filters the returned provider lists:
 - Matches the requested material against the provider's `provider_config.materials` array.
 - Sorts results prioritising local proximity (city/country match).
5. **[Backend]** Returns matching providers list.
6. **[Customer]** Selects a provider, configures shipping details, and clicks “Submit Print Order”.
7. **[Frontend]** Dispatches `POST /print/orders` with:
 - `provider_id`, `shipping_address`, `courier_name`, `items` (model IDs and specifications like layer heights or fill densities).
8. **[Database]** Looks up target provider and model records.
9. **[Backend]** Verifies the provider exists and is active.
 - If invalid: returns 400 Bad Request. Flow ends.
10. **[Database]** Inserts a new `Order` record:
 - `type = PRINT_JOB`
 - `status = PENDING`
 - `shipping_address` written as JSON
 - `items` created with `print_status = PENDING`
11. **[Backend]** Returns 201 Created.
12. **[Frontend]** Directs the user to the print order tracking page.

25.4 Decision Points Summary

Decision	Path/Condition	Outcome
Provider valid and active?	No	Return 400 Bad Request, stop
Proximity matches?	Yes	Bubble provider to the top of search list

25.5 Key Implementation Details

- **Print vs. Asset Orders:** In contrast to digital asset orders which immediately generate a Midtrans payment token, a `PRINT_JOB` order is saved in the database first to let the provider calculate custom pricing (shipping, handling, scaling adjustments) or accept/reject the job before final billing occurs.
- **Provider Specifications:** The provider configurations (capabilities, equipment, localized pricing) are stored in the database as a single structured `Json` field (`provider_config`), which makes it easy to add custom capabilities without needing migrations.

26 Activity Diagram - Profile Update

UML Type: Activity Diagram **Category:** Catalog **Source:** docs/diagrams/activity/catalog/activity_profile

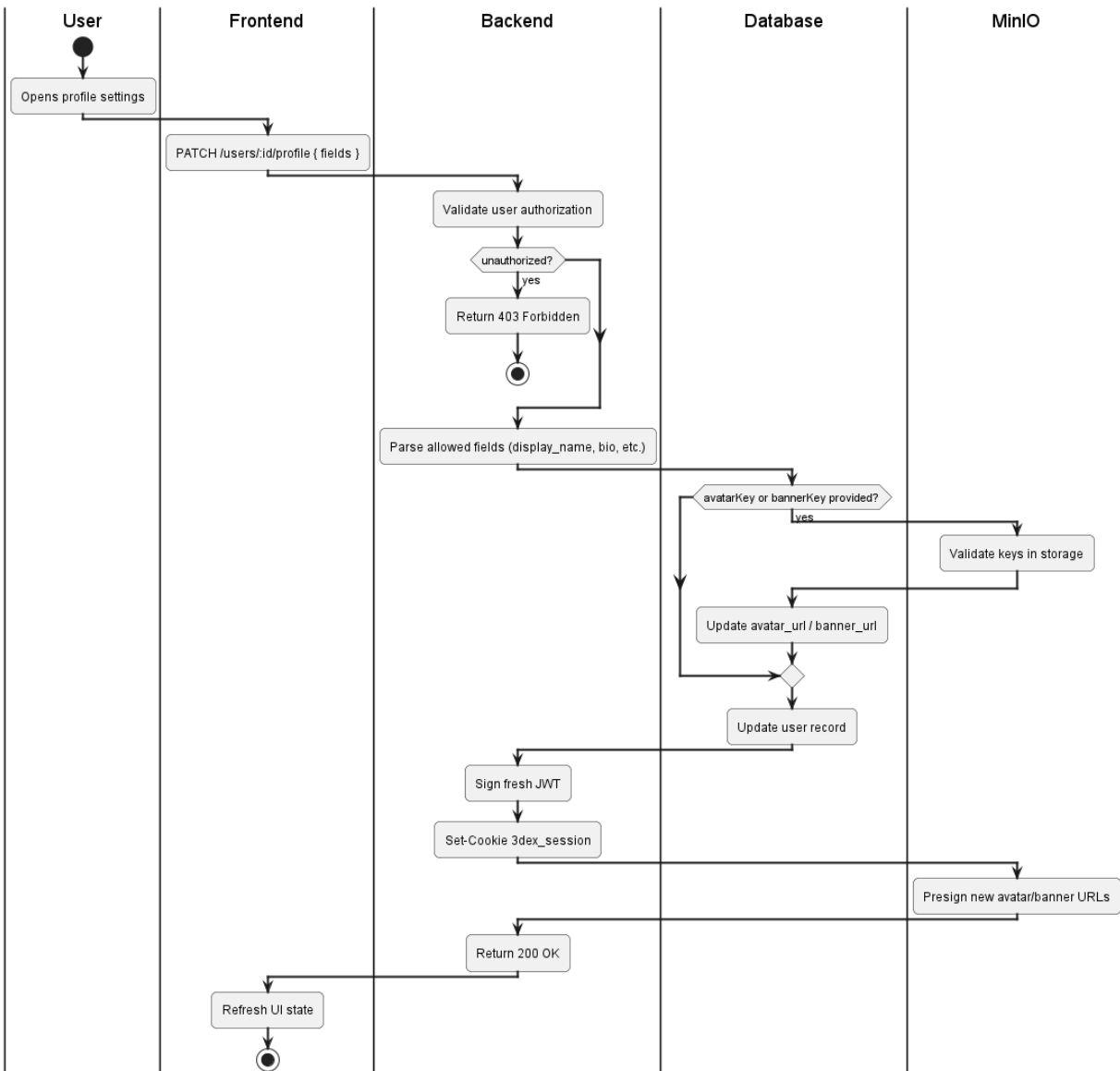


Figure 26: Activity Diagram - Profile Update

26.1 Overview

This activity diagram details the operational path when a user modifies their profile settings, including profile image uploads to S3 storage, JWT token re-signing, and session update coordination.

26.2 Swim Lanes

Lane	Responsibility
User	Modifies profile fields and saves settings
Frontend	Captures fields and dispatches request
Backend	Runs field sanitization, updates DB, and issues new tokens
Database	Updates User record details
MinIO	Validates uploaded asset keys and issues presigned URLs

26.3 Process Flow

1. [User] Opens their profile settings page.
2. [Frontend] Dispatches `PATCH /users/:id/profile` containing modified fields (display name, bio, banner keys, etc.).
3. [Backend] Verifies the request authentication and authorization context.
 - If unauthorized: returns 403 Forbidden. Flow ends.
4. [Backend] Parses and filters the input payload, extracting allowed fields.
5. [Backend] Checks if new S3 file keys (`avatarKey` or `bannerKey`) are provided.

Conditional Path: Image Update

- If image keys are present:
 1. [MinIO] Backend validates that the keys exist inside the storage bucket.
 2. [Database] Updates the user record with the new `avatar_url` and/or `banner_url` storage keys.
6. [Database] Persists all other changes to the `User` table.
 7. [Backend] Generates a fresh JWT containing updated profile summaries (display name, avatar URL, etc.).
 8. [Backend] Issues the fresh token in a new `3dex_session` Set-Cookie header.
 9. [MinIO] Generates updated presigned URLs for the user's avatar and banner image.
 10. [Backend] Returns 200 OK along with the updated user data.
 11. [Frontend] Updates its internal `AuthContext` state, refreshing the navigation bar and current view.

26.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authorized?	No	Return 403, stop
S3 keys provided?	Yes	Validate keys, update image URLs in database

26.5 Key Implementation Details

- **Sliding JWT Sessions:** When a user updates their profile, their session must reflect these updates immediately (e.g., in the navbar display name). Re-signing the JWT and returning a new session cookie prevents stale token states.
- **S3 Key Verification:** Rather than letting the user submit arbitrary strings for image paths, the backend calls MinIO storage APIs to verify the physical files exist before updating paths in the database.

27 Activity Diagram - User Registration

UML Type: Activity Diagram Category: Auth Source: docs/diagrams/activity/auth/activity_register.puml

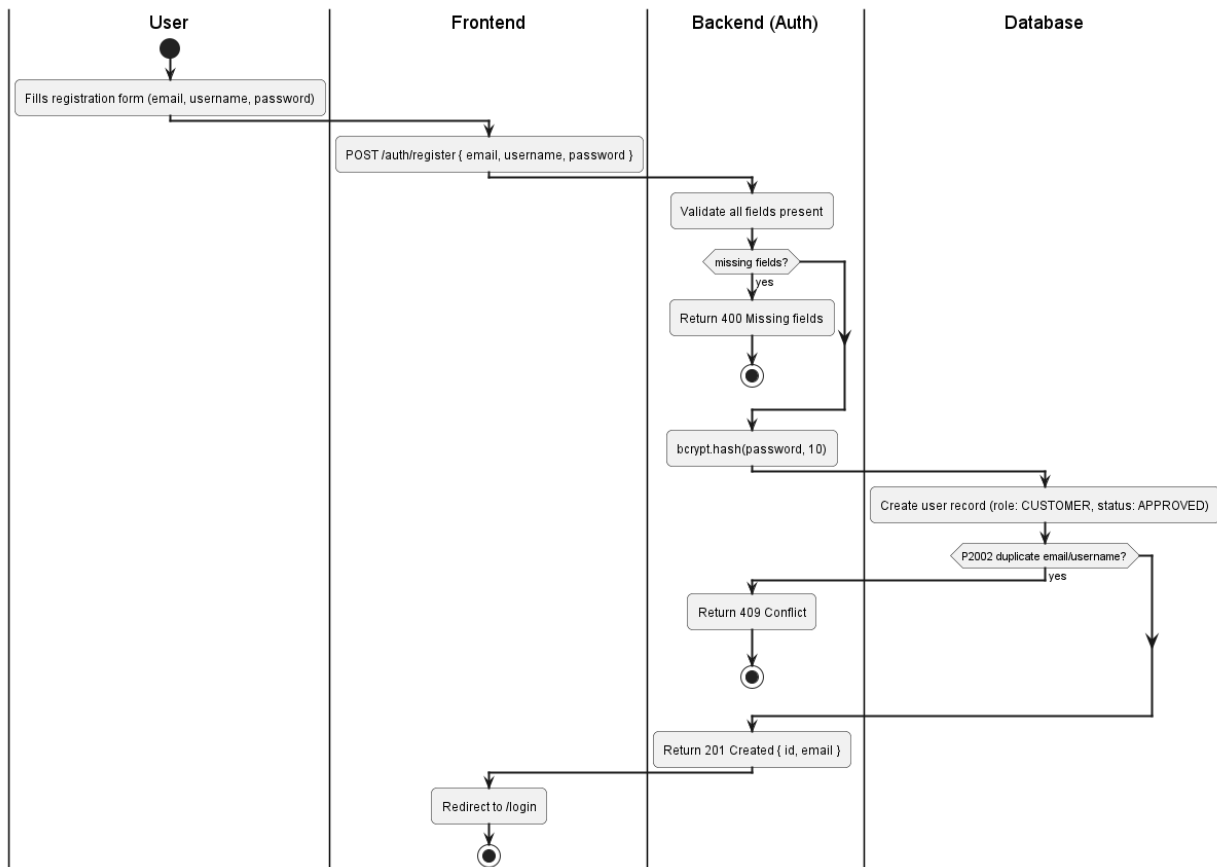


Figure 27: Activity Diagram - User Registration

27.1 Overview

This activity diagram shows the user registration process across swim lanes, emphasizing the validation steps, password hashing, and uniqueness enforcement.

27.2 Swim Lanes

Lane	Responsibility
User	Fills and submits the registration form
Frontend	Client-side validation
Backend	Server-side validation and persistence
Database	Data creation and constraint checking

27.3 Process Flow

1. **[User]** Fills in email, username, and password.
 2. **[Frontend]** Client validates that all fields are present and the username meets format rules (3-30 characters, alphanumeric, no spaces).
 - If invalid: shows field error. Flow ends.
 3. **[Frontend]** Sends `POST /auth/register { email, username, password }`.
 4. **[Backend]** Validates presence of all fields.
 - If missing: returns 400.
 5. **[Backend]** Calls `bcrypt.hash(password, 10)`.
 6. **[Database]** Calls `user.create({ email, username, password_hash, role: CUSTOMER, account_status: APPROVED })`.
 7. **[Database]** Checks unique constraints on `email` and `username`.
 - If Prisma error P2002 (unique violation): Backend returns 409 Conflict (“Email or username already taken”).
 8. **[Backend]** Returns 201 Created.
 9. **[Frontend]** Shows success message, redirects to `/login`.
-

27.4 Decision Points Summary

Decision	Yes Path	No Path
Client fields valid?	Continue	Show error, stop
Server fields present?	Continue	Return 400
DB unique constraint passes?	Continue	Return 409

27.5 Notes

- Registration creates only `CUSTOMER` accounts. Role upgrades require a separate application flow.
- No session token is issued at registration. The user must log in separately after completing registration.
- The username validation rules (3-30 chars, alphanumeric) are enforced at both the client level and should be enforced at the Prisma level via `@db.VarChar(30)`.

28 Activity Diagram - Report

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_report.puml

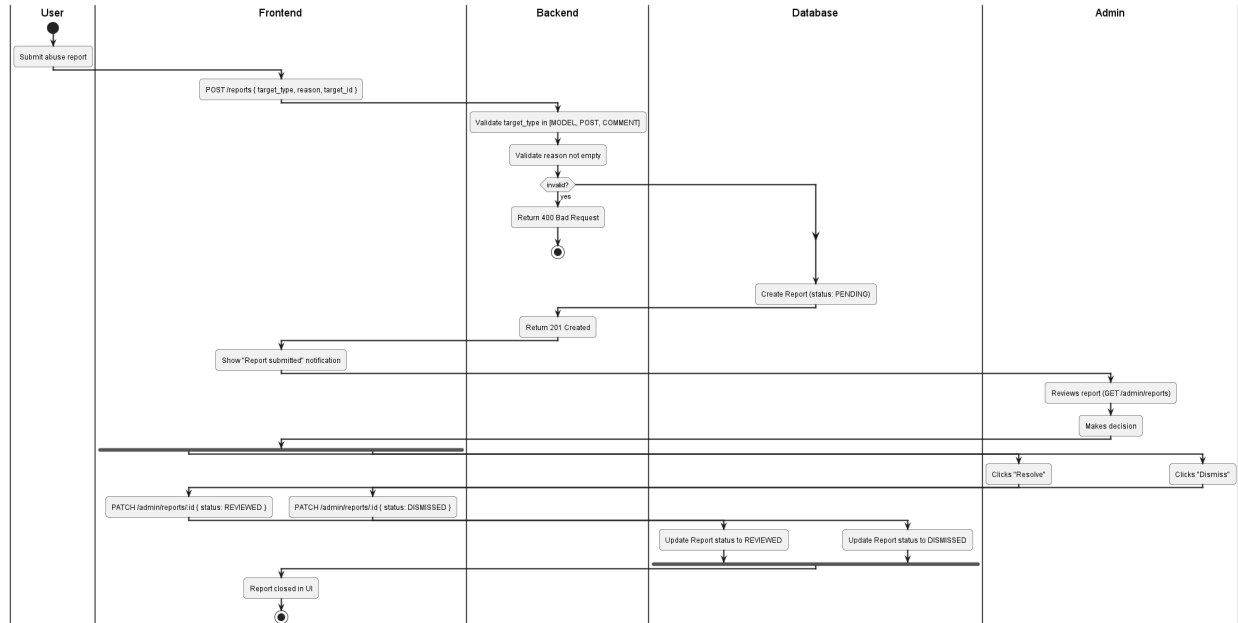


Figure 28: Activity Diagram - Report

28.1 Overview

This activity diagram shows the end-to-end report lifecycle from user submission to admin resolution, using parallel forks to represent the two resolution outcomes: Resolve or Dismiss.

28.2 Swim Lanes

Lane	Responsibility
User	Submits the abuse report
Frontend	Manages the report form and confirmation
Backend	Validates and persists
Database	Stores report and updates status
Admin	Reviews and resolves reports

28.3 Process Flow

User Submission Phase:

1. [User] Clicks “Report” on a model, post, or comment.
2. [Frontend] Sends POST /reports { target_type, reason, target_id }.
3. [Backend] Validates:
 - target_type must be in [MODEL, POST, COMMENT].
 - reason must not be empty.
 - If invalid: returns 400. Flow ends.
4. [Database] Creates Report { status: PENDING }.
5. [Backend] Returns 201 Created.
6. [Frontend] Shows “Report submitted” notification to user.

Admin Resolution Phase:

7. [Admin] Reviews the reports list at GET /admin/reports.
8. [Admin] Makes a resolution decision.

Fork:

Resolve path:

9A. [Admin] Clicks “Resolve”. 10A. [Frontend] Sends PATCH /admin/reports/:id { status: REVIEWED }. 11A. [Database] Updates Report.status = REVIEWED.

Dismiss path:

9B. [Admin] Clicks “Dismiss”. 10B. [Frontend] Sends PATCH /admin/reports/:id { status: DISMISSED }. 11B. [Database] Updates Report.status = DISMISSED.

Rejoin:

12. [Frontend] Report closed in UI.

28.4 Decision Points Summary

Decision	Yes Path	No Path
target_type valid?	Continue	Return 400
reason present?	Continue	Return 400
Admin resolves?	REVIEWED path	-
Admin dismisses?	DISMISSED path	-

28.5 Notes

- Report resolution does not automatically take action on the reported content. If the admin decides the report is valid, they must separately delete the content or ban the user.
- The Report_Status enum has three values: PENDING (submitted, awaiting review), REVIEWED (admin has seen and resolved it), and DISMISSED (admin has seen and rejected it as invalid).

29 Activity Diagram - Session Hydration

UML Type: Activity Diagram **Category:** Auth **Source:** docs/diagrams/activity/auth/activity_session_hydra

29.1 Overview

This activity diagram shows the session hydration process that runs on every page mount or refresh, determining whether the user’s session is still valid and populating the UI with fresh user data.

29.2 Swim Lanes

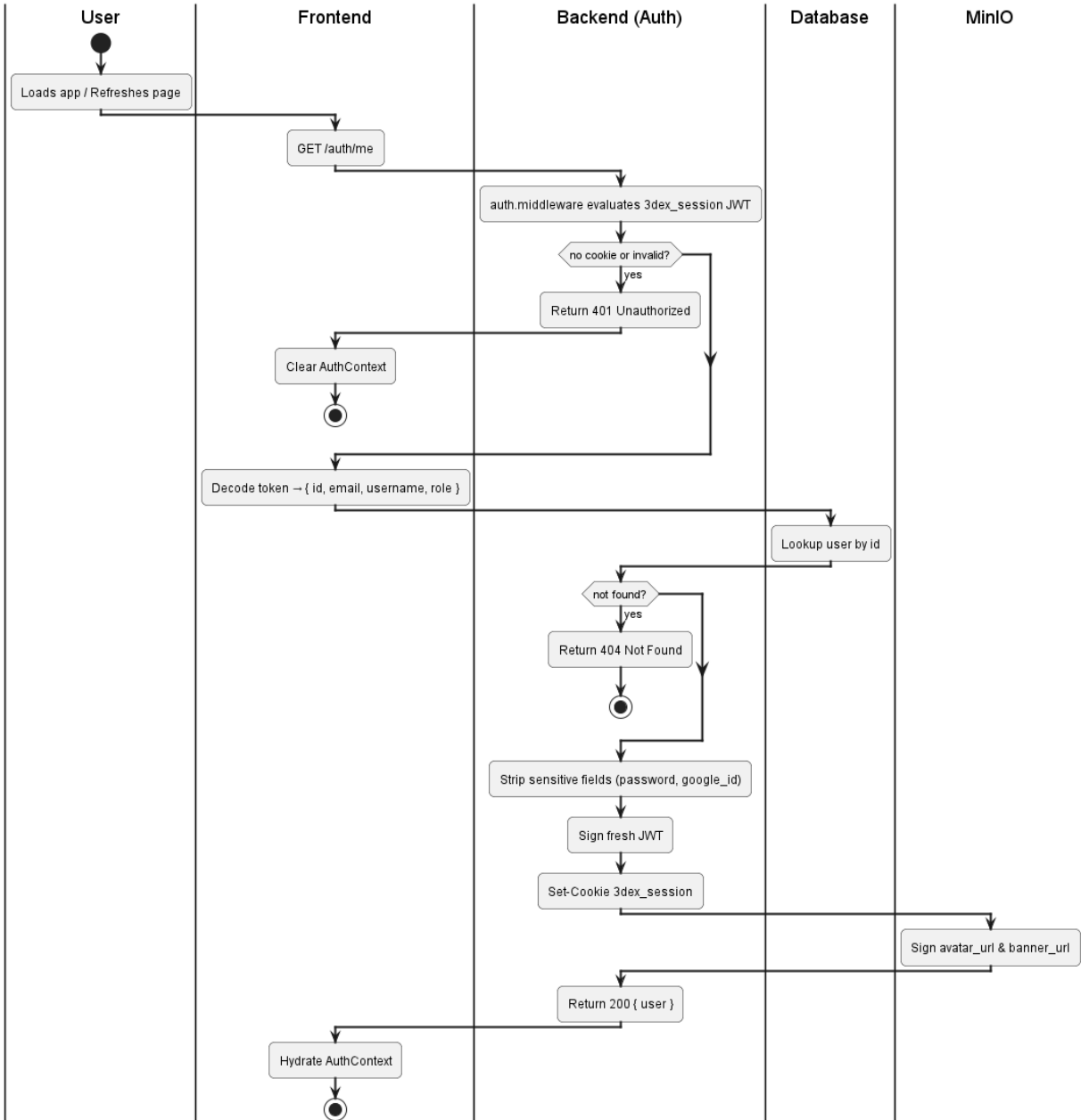


Figure 29: Activity Diagram - Session Hydration

Lane	Responsibility
User	Triggers the load by navigating to the app
Frontend	Initiates the hydration call on mount
Backend	Validates the cookie and fetches current user data
Database	Provides the current user record

29.3 Process Flow

1. [User] Opens or refreshes the app.
2. [Frontend] `useEffect` triggers `GET /auth/me` (with the `3dex_session` cookie).
3. [Backend] Auth middleware attempts to decode the JWT from the cookie.
 - If no cookie or invalid/expired token: returns 401. Frontend clears `AuthContext`, shows guest UI. Flow ends.
4. [Database] Fetches the user record by the ID in the JWT payload.
 - If user not found: returns 401 or 404. Frontend logs out. Flow ends.
5. [Backend] Strips sensitive fields (`password`, `google_id`) from the user object.
6. [Backend] Signs a fresh JWT and sets a new `3dex_session` cookie (sliding window).
7. [Backend] Calls MinIO to presign `avatar_url` and `banner_url`.
8. [Backend] Returns 200 with sanitized user + signed URLs.
9. [Frontend] Hydrates `AuthContext` with the user data.
10. [Frontend] Renders the authenticated UI state.

29.4 Decision Points Summary

Decision	Yes Path	No Path
Valid JWT in cookie?	Continue	Return 401, clear session
User exists in DB?	Continue	Return 401/404, logout

29.5 Notes

- This flow runs on every app load, not just login. This is by design to ensure the UI always reflects the current server state (e.g., if an admin has suspended the user's account).
- The token refresh on each hydration call extends the session, implementing a sliding-window expiry.
- Only public-safe fields are returned. `password` and `google_id` are never sent to the browser.

30 Activity Diagram - Stats Aggregation

UML Type: Activity Diagram **Category:** Admin **Source:** docs/diagrams/activity/admin/activity_stats_aggr

30.1 Overview

This activity diagram describes the background stats aggregation process, which computes platform-level metrics for a defined time period and stores them in the `Stats` table for dashboard display.

6. **[Database]** Creates a new **Stats** record: { **period_start**, **period_end**, **data** }.
 7. **[Backend (Cron Service)]** Returns the new **Stats** record.
 8. **[Frontend]** Admin dashboard chart reflects the updated data on next load.
-

30.4 Decision Points Summary

There are no conditional branches in this flow. The process is always executed fully once triggered.

30.5 Notes

- The parallel fork in the Database lane shows that multiple aggregation queries should ideally run concurrently. In practice, Prisma does not natively parallelize queries in a transaction; a **Promise.all** wrapper would be needed.
- Each **Stats** record is a timestamped snapshot. Multiple records accumulate over time, providing a history of platform metrics. The dashboard displays the most recent record and can chart the history.
- The **data** JSON field structure is application-defined and would include fields like { **total_orders**, **total_revenue**, **new_users**, **new_models**, **top_models**: [], **top_artists**: [] }.
- The cron trigger interval is not specified in the diagram. A reasonable interval for a marketplace would be daily or weekly aggregation.

31 Activity Diagram - User Review

UML Type: Activity Diagram **Category:** Social **Source:** docs/diagrams/activity/social/activity_user_review

31.1 Overview

This activity diagram describes the peer-to-peer user review process (rating/reviewing an Artist or Provider directly). It details eligibility verification checks (requiring a prior completed transaction) and automatic profile rating aggregation updates.

31.2 Swim Lanes

Lane	Responsibility
Customer	Accesses profile and submits reviews
Frontend	Checks eligibility status and renders forms
Backend	Processes transaction validation and compiles ratings
Database	Queries purchases, print orders, and aggregates scores

31.3 Process Flow

31.3.1 Phase 1: Eligibility Check

1. **[Customer]** Navigates to an artist or provider's profile page.
2. **[Frontend]** Dispatches **GET /users/:id/reviews/eligibility** with credentials.

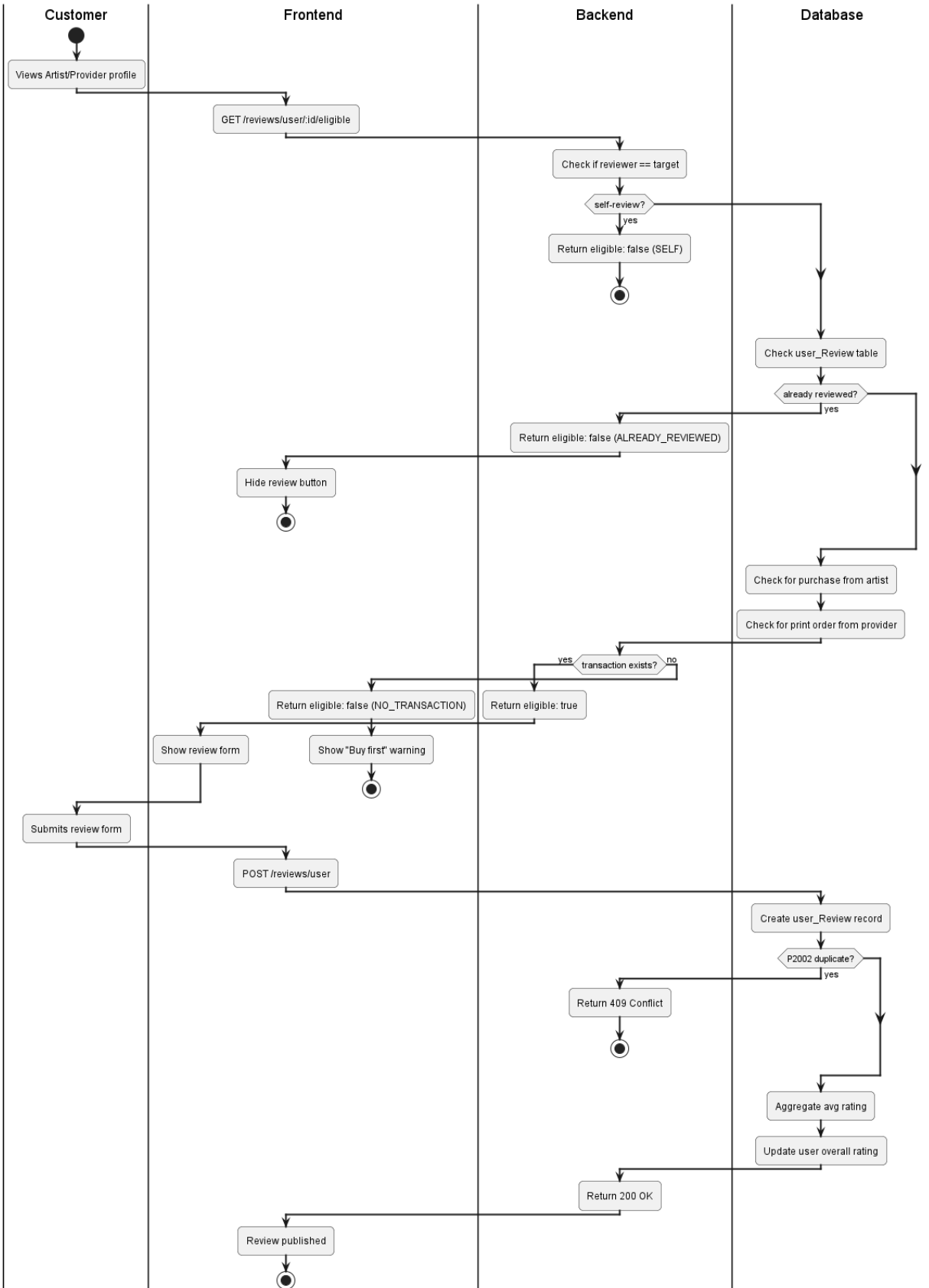


Figure 31: Activity Diagram - User Review

3. [Database] Queries `User_Review` to check if the caller has already reviewed this user.
 - **If review already exists:** Backend returns { `eligible: false, reason: "ALREADY_REVIEWED"` }. Frontend hides form. Flow ends.
 4. [Database] Queries transaction tables:
 - For an **Artist**: Checks for a **Purchase** record of any model belonging to this artist.
 - For a **Provider**: Checks for a paid **Order** of type `PRINT_JOB` assigned to this provider.
 5. [Backend] Evaluates transaction presence.
 - If no transaction exists: returns { `eligible: false, reason: "NO_TRANSACTION"` }. Frontend displays block notice. Flow ends.
 - If a transaction exists: returns { `eligible: true` }. Frontend renders review form.
-

31.3.2 Phase 2: Review Submission

6. [Customer] Fills out rating (1-5) and comment, and clicks submit.
 7. [Frontend] Dispatches `POST /users/:id/reviews` containing { `rating, comment` }.
 8. [Backend] Verifies the rating is an integer between 1 and 5.
 9. [Backend] Verifies `reviewer_id != target_user_id` (preventing self-reviews).
 - If invalid: returns 400. Flow ends.
 10. [Database] Inserts the `User_Review` record.
 11. [Database] Runs aggregation query: recalculates total review counts and average rating for the target user.
 12. [Database] Updates the target `User` record with the new `rating` and `review_count` scores.
 13. [Backend] Returns 200 OK along with the new review record.
 14. [Frontend] Appends the review to the public profile reviews list.
-

31.4 Decision Points Summary

Decision	Path/Condition	Outcome
Already reviewed?	Yes	Return eligible: false, stop
Completed transaction?	No	Return eligible: false, stop
Self-review?	Yes	Return 400 Bad Request, stop

31.5 Key Implementation Details

- **Peer-to-Peer vs. Product Reviews:** Peer-to-peer ratings are stored in the `User_Review` table and aggregate onto the target's `User` record, whereas model reviews are stored in the `Review` table and aggregate onto the `Model` record.
- **Transaction Gate:** To prevent rating manipulation, reviews are restricted. Users must buy a 3D model from an artist or complete a physical print job with a provider before they are eligible to rate them.

32 Activity Diagram - Wishlist Toggle

UML Type: Activity Diagram **Category:** Commerce **Source:** docs/diagrams/activity/commerce/activity_wish

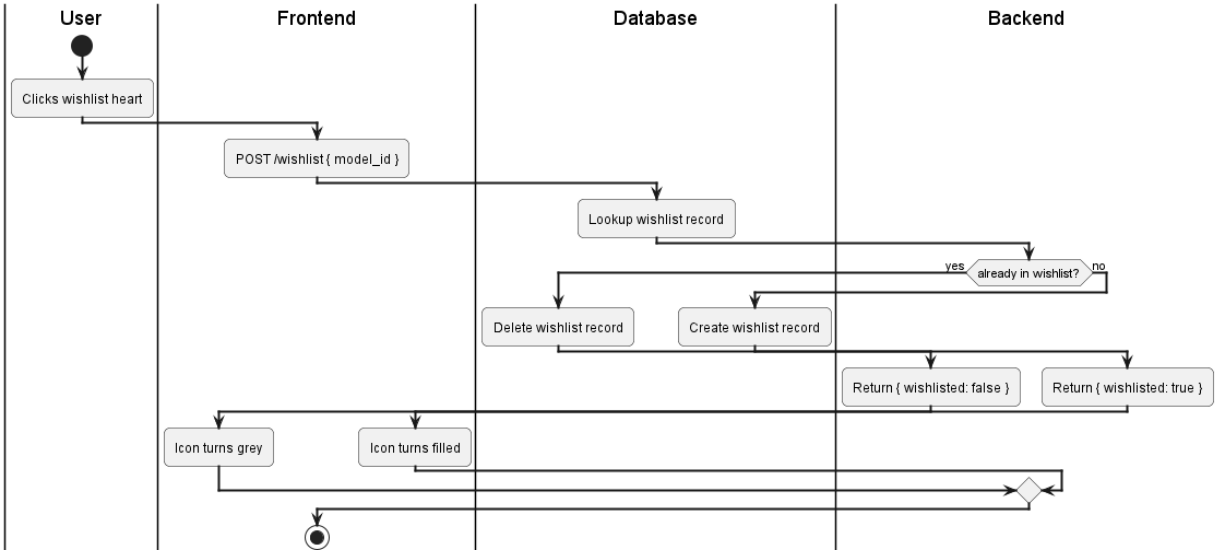


Figure 32: Activity Diagram - Wishlist Toggle

32.1 Overview

This activity diagram describes the dual-path logic of wishlisting a model. It covers how a single heart click checks current status and branches to either create (add) or delete (remove) the wishlist record.

32.2 Swim Lanes

Lane	Responsibility
User	Clicks the heart icon
Frontend	Issues the toggle request and handles optimistic UI
Backend	Verifies session and routes toggle action
Database	Performs lookup, insert, or delete operations

32.3 Process Flow

1. [User] Clicks the “Wishlist” (heart) icon on a model.
2. [Frontend] Dispatches `POST /wishlist/toggle` with `{ model_id }`.
3. [Backend] Verifies the requesting user is authenticated.
 - If not authenticated: returns 401 Unauthorized. Flow ends.
4. [Database] Queries `Wishlist` table for a record matching the `(user_id, model_id)` unique pair.
5. [Backend] Evaluates the check:

Decision Branch: Exist or Not?

- **Path A: Record Exists (Remove from Wishlist)**
 1. [Database] Deletes the `Wishlist` record.
 2. [Backend] Returns 200 OK with `{ wished: false }`.
 3. [Frontend] Changes heart icon to outline.
- **Path B: Record Does Not Exist (Add to Wishlist)**

1. **[Database]** Inserts a new `Wishlist` record.
 2. **[Backend]** Returns 200 OK with `{ wished: true }`.
 3. **[Frontend]** Changes heart icon to filled.
-

32.4 Decision Points Summary

Decision	Path/Condition	Outcome
Authenticated?	No	Return 401, stop
Record exists?	Yes	Delete wishlist record, return <code>wished: false</code>
Record exists?	No	Create wishlist record, return <code>wished: true</code>

32.5 Key Implementation Details

- **Atomic Unique Constraint:** The database has a unique index on `(user_id, model_id)`. Even in highly concurrent environments, this prevents duplicate entries.
- **Toggle Endpoint Pattern:** Rather than exposing separate `POST` (add) and `DELETE` (remove) endpoints, the `/wishlist/toggle` endpoint simplifies frontend state management by returning the new status as a boolean, allowing the client to adjust its styling dynamically.